

On a Grover-based quantum signature scheme and a teleportation-based design

Guoliang Xu ^{1*}, Songyao Xue ¹, Xiangfu Zou ^{2,*} and Yumei Zhang ³

¹ School of Software, Luoyang Normal University, Luoyang, 471934, China; xu1guo2liang@foxmail.com

² School of Mathematics and Computational Science, Wuyi University, Jiangmen, 529020, China

³ Conservatory of Music, Luoyang Normal University, Luoyang, 471934, China

* Correspondence: xu1guo2liang@foxmail.com, xf.zou@hotmail.com

Abstract

Quantum computation, with tools including Grover's algorithm, quantum walks, and quantum teleportation, plays an important role in quantum signature designs. Although such designs offer signature functionality, they sometimes come at the cost of security. This paper first reviews a Grover-based scheme and shows how man-in-the-middle attacks enable complete key recovery, Alice's disavowal, and Bob's forgery. We then propose an arbitrated quantum signature scheme based on quantum teleportation and a strengthened quantum one-time pad. The security of the proposed scheme is analyzed through a formal adversarial model for unforgeability and non-repudiation, and its resilience against replay, intercept-and-resend, man-in-the-middle, entanglement, and collective or coherent attacks is examined. Numerical simulations under depolarizing noise and an asymptotic resource analysis further validate the practicality of the scheme.

Keywords: Quantum computation; Grover algorithm; Quantum signature; Security analysis

1. Introduction

In the field of quantum computation and communication, Grover's algorithm, quantum teleportation and quantum walk have inspired a variety of cryptographic constructions far beyond their original application scenarios [1–7]. Leveraging quantum superposition, phase manipulation and iterative evolution, these algorithm-driven schemes feature concise and flexible architecture, and offer security guarantees based on quantum information-theoretic principles. Among these quantum primitives, Grover's algorithm, as a fundamental building block in quantum computation and complexity theory, is particularly intriguing and conceptually appealing, offering a novel perspective for the design and construction of quantum signature schemes.

Among quantum signature paradigms, arbitrated quantum signature (AQS) schemes introduce a trusted or semi-trusted arbitrator to assist verification and dispute resolution [8–10]. Since the early scheme of Zeng and Keitel [8], a range of AQS constructions have been proposed using entanglement, Bell states, or quantum one-time pad (QOTP) encryption [9, 11–13]. A number of schemes built from seemingly sound quantum primitives were later shown to admit forgery, repudiation, or impersonation attacks [14–16]. These results suggest that the security of quantum signature schemes depends not only on the local soundness of the primitives employed, but also on the way those primitives are combined at the protocol level.

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Entropy* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

In 2015, Yoon et al. [4] proposed a quantum signature scheme that adopts Grover-type operations for signature generation and verification. Later in 2023, Yoon et al. [6] further extended this line of research by presenting a quantum asymmetric key cryptographic scheme based on Grover iteration. Their work migrated Grover-related operations from quantum signature scenarios to the domain of quantum asymmetric encryption, further enriching the application boundaries of Grover primitives in quantum cryptography. From the algorithmic perspective, such designs are conceptually appealing since they embed standard quantum search routines directly into cryptographic workflows. Nevertheless, combining reversible quantum transformations with publicly accessible verification parameters leads to an inherent structural conflict: information that is harmless in standard quantum search scenarios may become a critical security vulnerability in signature and encryption systems.

In this paper, we study quantum signature security from that structural perspective. Our main contributions are as follows:

- (1) We analyze the Grover-based quantum signature scheme of Yoon *et al.*[4] and demonstrate that it is vulnerable to man-in-the-middle attacks, which enable complete key recovery, Alice’s disavowal, and Bob’s existential forgery.
- (2) We propose an arbitrated quantum signature scheme based on quantum teleportation and a strengthened QOTP [16]. A formal security analysis under a well-defined adversarial model shows that the scheme satisfies unforgeability and non-repudiation, and numerical simulations confirm its practical viability.

The remainder of this paper is organized as follows. Section 2 introduces the necessary preliminaries. Section 3 reviews and analyzes the Grover-based quantum signature scheme of Yoon *et al.*[4]. Section 4 proposes and analyzes an arbitrated quantum signature scheme that combines quantum teleportation and strengthened QOTP [16]. Section 5 provides a resource overhead analysis of the proposed scheme and YKLY’2015, and simulations of the attack of the proposed scheme. Finally, Section 6 concludes the paper. The complete simulation codes are provided in the Appendix.

2. Preliminaries

This section introduces the notation, quantum primitives, and background used throughout the paper.

To begin with, for the reader’s convenience, the principal symbols used throughout the paper are summarized in Table 1.

Table 1. Summary of key notation.

General		Primitives		Protocol	
$\mathbb{C}$	Complex numbers	$\delta_x, \delta_y, \delta_z$	Pauli operators	$K_{AT}(16)$	Key (Alice-Trent)
i	Imaginary unit	$\bar{F}$	State fidelity	$K_{BT}(24)$	Key (Bob-Trent)
$\cong$	Equal up to phase	$ p\rangle$	Message state	$K'_{AT}(4)$ etc.	Key segments
α	Physical error rate	$E_K p\rangle$	Strengthened QOTP	X_1, X_2, X_3	Bell outcomes
λ	Security parameter	$E_K(\rho)$	Standard QOTP	$ p_1\rangle, p_2\rangle, p_3\rangle$	Message copies
K^i	i -th bit of K	$U(X)$	Pauli operation	$ p'_2\rangle, p'_3\rangle, X'_1$	Forged values
$ X\rangle$	State for n -bit X	$ \varphi^\pm\rangle, \psi^\pm\rangle$	Bell states	p_{dep}	Depolarizing noise
$t_{\text{comm}}, t_{\text{op}}$	Latency, local time	$ \varphi^+\rangle_{AB}, \varphi^+\rangle_{BT}$	Shared Bell pairs		

Then, Grover’s algorithm provides a quadratic speedup for unstructured search by iteratively amplifying the amplitude of marked states [5]. In Grover-based quantum signature schemes [4], the mechanism of unitary operations is represented by $U_S = I - 2|K\rangle\langle K|$, $U_V = 2|M\rangle\langle M| - I$, and their combined action $U_V U_S |M\rangle = a|K\rangle$. Here, applying the unitary operators U_S and U_V , this mechanism can be used to identify the desired data $|K\rangle$ (for example $|K\rangle = |00\rangle$) from database $|M\rangle$ (for example, $|M\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle +$

$|11\rangle\rangle$). The Grover-based quantum signature scheme [4] was proposed by utilizing the unitary operations U_V and U_S .

The teleportation procedure used in our protocol proceeds as follows. Alice combines the message state $|p\rangle = \alpha|0\rangle + \beta|1\rangle$ ($\alpha, \beta \in \mathbb{C}$, $|\alpha|^2 + |\beta|^2 = 1$) with the Bell state $|\varphi^+\rangle_{AB}$ to obtain a three-particle entangled state

$$|p\rangle_A \otimes |\varphi^+\rangle_{AB} = \frac{1}{2} [|\varphi^+\rangle_{AA}(\alpha|0\rangle + \beta|1\rangle)_B + |\varphi^-\rangle_{AA}(\alpha|0\rangle - \beta|1\rangle)_B + |\psi^+\rangle_{AA}(\alpha|1\rangle + \beta|0\rangle)_B + |\psi^-\rangle_{AA}(\alpha|1\rangle - \beta|0\rangle)_B]. \quad (1)$$

Defining $U(X)$ as

$$U(X) = \begin{cases} I & = |0\rangle\langle 0| + |1\rangle\langle 1|, X = 00, \\ \delta_x & = |1\rangle\langle 0| + |0\rangle\langle 1|, X = 01, \\ \delta_z & = |0\rangle\langle 0| - |1\rangle\langle 1|, X = 10, \\ \delta_z\delta_x & = |0\rangle\langle 1| - |1\rangle\langle 0|, X = 11. \end{cases} \quad (2)$$

the above state $|p\rangle_A \otimes |\varphi^+\rangle_{AB}$ can be compactly written as

$$|p\rangle_A \otimes |\varphi^+\rangle_{AB} = \frac{1}{2} \sum_{X=00}^{11} [I \otimes U(X)] |\varphi^+\rangle_{AA} \otimes U(X) |p\rangle_B, \quad (3)$$

where each $[I \otimes U(X)] |\varphi^+\rangle$ is one of the four Bell states. Alice then performs a Bell measurement on her two particles. The state collapses to

$$[I \otimes U(X)] |\varphi^+\rangle_{AA} \otimes U(X) |p\rangle_B, \quad (4)$$

with X uniformly distributed over $\{00, 01, 10, 11\}$. Alice obtains X from the Bell measurement outcome, while Bob's particle collapses to $U(X)|p\rangle$. We refer to this process as Alice teleporting $U(X)|p\rangle$ to Bob.

The strengthened quantum one-time pad (QOTP) proposed by Kim *et al.* [16] addresses forgeability issues present in standard QOTP-based arbitrated quantum signature schemes [14,15]. Using a 4-bit key K , a single-qubit message $|p\rangle = \alpha|0\rangle + \beta|1\rangle$ is encrypted as

$$|q\rangle = E_K |p\rangle = \delta_x^{K^1} \delta_z^{K^2} T \delta_x^{K^3} \delta_z^{K^4} |p\rangle, \quad (5)$$

where $T = \frac{i}{\sqrt{3}}(\delta_x - \delta_y + \delta_z)$. The standard QOTP achieves information-theoretic security when keys are uniformly random and never reused [17,18]. Our protocol inherits this property by generating fresh keys for each execution.

3. Analysis of a Grover-based quantum signature scheme (YKLY'2015)

In this section, we analyze the quantum signature scheme based on Grover's search algorithm proposed by Yoon, Kang, Lim, and Yang (YKLY'2015) [4]. We show that the scheme suffers from fundamental structural vulnerabilities that lead to complete key recovery and universal forgery.

3.1. Review of a Grover-based quantum signature scheme (YKLY'2015)

In the interest of readability, we briefly review YKLY'2015 [4] in this section. The scheme consists of two communication processes: between Alice and TC, and between TC and Bob. Meanwhile, both of the two processes consist of three phases: preparation, signature and verification. YKLY'2015 [4] can be described as follows.

3.1.1. Process between Alice and TC

In the Preparation phase, there are the following two steps.

(AT1) Alice announces the classical message m that she wants to send to Bob on the public board. Then, Alice and TC share the secret key K_{AT} by a QKD protocol, such as the QKD protocols [19–21], which provides unconditional security [22,23].

(AT2) Alice creates the signing unitary operator $U_{S_{AT}} = I - 2|K_{AT}\rangle\langle K_{AT}|$ where $|K_{AT}\rangle$ is a two-qubit state generated from the classical two-bit key K_{AT} . For example, $|K_{AT}\rangle = |ij\rangle$ if $K_{AT} = ij$ for all $i, j \in \{0, 1\}$.

In the Signature phase, Alice makes her signature in Step (AT3) and sends it to TC by Steps (AT4)–(AT7).

(AT3) Using the classical message m , Alice generates an original message qubit $|M\rangle$ where $|M\rangle = |-\rangle \otimes |-\rangle$ if $m = 00$, $|M\rangle = |-\rangle \otimes |+\rangle$ if $m = 01$, $|M\rangle = |+\rangle \otimes |-\rangle$ if $m = 10$, and $|M\rangle = |+\rangle \otimes |+\rangle$ if $m = 11$. Then, she obtains the signature $U_{S_{AT}}|M\rangle = (I - 2|K_{AT}\rangle\langle K_{AT}|)(|M\rangle)$ using $|M\rangle$ and $U_{S_{AT}}$.

(AT4) Alice includes decoy qubits of the EPR pair, $|\psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$, in random locations among the message qubits $U_{S_{AT}}|M\rangle$. Alice sends TC the first sequence of particles of the message qubits, and tells TC the locations of half the decoy qubits.

(AT5) TC performs measurement on the decoy qubits using Z or X basis, and tells Alice his measurement bases and measurement outcomes. Alice uses the same measuring bases to measure the corresponding qubits in the sequence and compares the measurement outcomes with TC's. If the error rate is smaller than the channel average error rate, Alice and TC conclude that there are no eavesdroppers in the channel and continue the protocol.

(AT6) Alice performs random unitary operations chosen from $\{I, X, Y, Z\}$ on the other half of the decoy qubits. She sends the other sequence of the particles to TC, and informs TC the positions of decoy qubits and the types of unitary operations she has chosen.

(AT7) TC confirms the security of the communication channel based on the received information.

In the Verification phase, there is only one step.

(AT8) TC first makes the verifying unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message m . Then, TC implements U_V on the received message to recover the shared key K_{AT} as $U_V U_{S_{AT}}|M\rangle = \pm|K_{AT}\rangle$. Finally, TC compares the result of $U_V U_{S_{AT}}|M\rangle$ with the shared key. If they are identical, TC confirms that Alice is a legitimate signer.

3.1.2. Process between TC and Bob

Confirming that the received message was legitimately signed by Alice, TC sends the message, his signature, to Bob by repeating the same processes above. By this process, Bob can ensure that the message received from Alice, in fact, the channel, is signed by her.

In the Preparation phase, there are the following two steps.

(TB1) TC and Bob share the secret key K_{TB} by a QKD protocol, such as the QKD protocols [19–21], which can provide unconditional security [22,23].

(TB2) TC makes the signing unitary operator $U_{S_{TB}} = I - 2|K_{TB}\rangle\langle K_{TB}|$ where $|K_{TB}\rangle$ is a two-bit state generated from K_{TB} as with K_{AT} .

In the Signature phase, TC makes his signature in Step (TB3) and sends it to Bob by Steps (TB4)–(TB7).

(TB3) TC signs the message $|M\rangle$ using $U_{S_{TB}}$ as $U_{S_{TB}}|M\rangle = (I - 2|K_{TB}\rangle\langle K_{TB}|)(|M\rangle)$.

(TB4) TC includes decoy qubits of the EPR pair, $|\psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle)$, in random locations among the message qubits $U_{S_{TB}}|M\rangle$. TC sends Bob the first sequence of particles of the message qubits, and informs Bob of the locations of half the decoy qubits.

(TB5) Bob performs measurement on the decoy qubits using Z or X basis, and tells TC his measurement bases and measurement outcomes. TC uses the same measuring bases to

measure the corresponding decoy qubits in the sequence and compares the measurement outcomes with those of Bob. If the error rate is smaller than the channel average error rate, TC and Bob conclude that there are no eavesdroppers in the channel, and continue the protocol.

(TB6) TC performs unitary operations chosen from $\{I, X, Y, Z\}$ on the other half of the decoy qubits. TC sends the other sequence of particles to Bob, and informs him the positions of decoy qubits and the types of unitary operations he has chosen.

(TB7) Bob confirms the security of communication channel based on the information. In the Verification phase, there is only one step.

(TB8) Bob makes the verifying unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message m . And then, Bob performs U_V on the received message to recover the shared key K_{TB} as $U_V U_{S_{TB}}|M\rangle = \pm|K_{TB}\rangle$. Bob compares the result of $U_V U_{S_{TB}}|M\rangle$ with the shared key. If they are identical, Bob confirms that TC is a legitimate signer.

3.2. Man-in-the-middle (MITM) attacks

In YKLY'2015 [4], TC plays a crucial role including the arbitrator. When a dispute arises between Alice and Bob, only TC has the right to dispute resolution. In this section, we first give a MITM attack on YKLY'2015 [4]. Then, we show how Alice uses the proposed MITM attack to disavow signature and Bob uses the proposed MITM attack to forge signature.

3.2.1. Eve's MITM attack

The MITM attack on YKLY'2015 [4] consists of three processes: between Alice and Eve, between Eve and TC, and between TC and Bob. In this attack, Eve completely intercepts and replaces the quantum channel between Alice and TC, impersonating TC to Alice and Alice to TC. The attack is illustrated in Figure 1 and we present it in detail as follows.

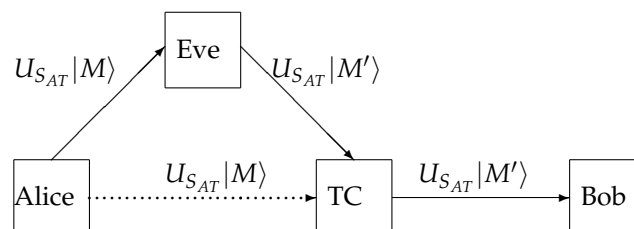


Figure 1. MITM attack. Solid lines correspond to the process of the attack while the dotted line denotes the corresponding process of YKLY'2015 [4].

In the Process between Alice and Eve, the attacker Eve first intercepts the message qubits $U_{S_{AT}}|M\rangle$ to obtain the shared key K_{AT} by pretending to be TC and performing the following four steps.

(AE1) In Step (AT4), Eve intercepts the first sequence of particles of the message qubits $U_{S_{AT}}|M\rangle$ and the message that Alice tells TC the locations of half the decoy qubits.

(AE2) In Step (AT5), Eve performs measurement on the decoy qubits using Z or X basis, and impersonates TC to tell Alice her measurement bases and measurement outcomes.

(AE3) In Step (AT6), Eve intercepts the other sequence of the particles that Alice sends to TC and the message that Alice tells TC the positions of decoy qubits and the types of unitary operations.

(AE4) In Step (AT8), Eve makes the unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message. Then, she implements U_V on the received message to recover the shared key as $U_V U_{S_{AT}}|M\rangle = \pm|K_{AT}\rangle$. Thereby, Eve can obtain the shared key K_{AT} successfully.

In the Process between Eve and TC, Eve performs the following two steps to pass TC's verify by posing as Alice.

(ET1) Eve pretends to be Alice to announce forged message m' and makes its signature $U_{S_{AT}}|M'\rangle$ with K_{AT} .

(ET2) Eve impersonates Alice to communicate with TC by performing Steps (AT4)-(AT8).

Finally, there is nothing Eve needs to do in the process between TC and Bob.

By performing the MITM attack above, $U_{S_{AT}}|M'\rangle$ as the signature of the forged message m' can pass TC's verifying. Therefore, Eve can forge Alice's signature for any message successfully.

Remark 1. Similarly, to forge Alice's signature in YKLY'2015 [4], Eve can perform an MITM attack between TC and Bob: Eve intercepts the message qubits $U_{S_{TB}}|M\rangle$ to obtain the shared key K_{TB} , pretends to be Alice to announce forged message m' , makes the forged signature $U_{S_{TB}}|M'\rangle$ with K_{TB} , and sends $U_{S_{TB}}|M'\rangle$ to Bob as TC in YKLY'2015.

3.2.2. Alice's disavowal attack

Alice can achieve her disavowal in YKLY'2015 [4] by performing an MITM attack between TC and Bob which consists of two processes: between TC and Alice, and between Alice and Bob. The MITM attack is described in Figure 2. Its details will be shown in the

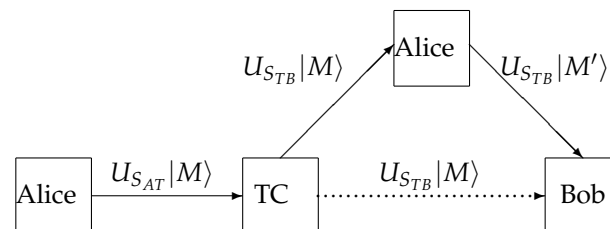


Figure 2. Alice's disavowal attack. Solid lines correspond to the process of the attack while the dotted line denotes the corresponding process of YKLY'2015 [4].

following.

In the Process between TC and Alice, Alice first pretends to be Bob and performs the following four steps.

(TA1) In Step (TB4), Alice intercepts the first sequence of particles of the message qubits $U_{S_{TB}}|M\rangle$ and the message that TC tells Bob the locations of half the decoy qubits.

(TA2) In Step (TB5), Alice performs measurement on the decoy qubits using Z or X basis, and impersonates Bob to tell TC her measurement bases and measurement outcomes.

(TA3) In Step (TB6), Alice intercepts the other sequence of the particles that TC sends to Bob and the message that TC tells Bob the positions of decoy qubits and the types of unitary operations.

(TA4) In Step (TB8), Alice makes a unitary operator $U_V = 2|M\rangle\langle M| - I$ based on the known message. She implements the operator U_V on the received message to recover the shared key as $U_V U_{S_{TB}}|M\rangle = \pm|K_{TB}\rangle$. So far, Alice has got the shared key K_{TB} .

In the Process between Alice and Bob, Alice poses as TC to perform the following two steps to let Bob accept her forged signature.

(AB1) Alice announces the forged message m' and makes the forged TC's signature $U_{S_{TB}}|M'\rangle$ with K_{TB} .

(AB2) Alice communicates with Bob to perform Steps (TB4)-(TB8) by posing as TC.

By performing operations above, Alice can forge a message and its signature which can pass Bob's verifying. Note that the sources of the messages received by Bob cannot be distinguished. In fact, the message which TC has verified and the message which Bob has received are not the same one in Alice's attack. As a result, Alice can announce that the m is not the initial message to achieve her benefit.

3.2.3. Bob's forgery attack

Bob's forgery attack follows the same pattern as Eve's MITM attack, applied to the TC-Bob phase. As the legitimate receiver, Bob already possesses the correct position to intercept the message qubits $U_{S_{TB}}|M\rangle$ sent from TC to Bob in Steps (TB4)-(TB7). Bob extracts the shared key K_{TB} using the publicly known verifying operator $U_V = 2|M\rangle\langle M| - I$, which he can construct from the message m announced on the public board. With K_{TB} in hand, Bob can sign any forged message m' by preparing $U_{S_{TB}}|M'\rangle$ and present it as evidence that the message originated from Alice and was verified by TC. Since TC's verification in the TC-Bob phase only confirms that a message was signed with K_{TB} , Bob's forged signature passes verification. Consequently, Bob can successfully forge Alice's signature for any message of his choice.

3.3. The main reasons why YKLY'2015 is insecure

In a signature scheme, TC should be able to judge the validity of the signature and arbitrate the dispute between Alice and Bob. However, by the attacks in Section 3.2, it is not the case in YKLY'2015 [4]. In this section, we discuss the reasons why YKLY'2015 [4] is vulnerable to MITM attacks.

First, according to YKLY'2015 [4], we can know that the process between Alice and TC is used to confirm that the received message m was legitimately signed by Alice or not. That is, in Step (AT1), TC cannot confirm that the message in public board (such as the initial message m and the forged message m') really comes from Alice or not. Otherwise, the subsequent steps in the scheme [4] are unnecessary because the message in public board has been authenticated by TC. In other words, the public board cannot provide authentication so that Eve can announce some messages in public board.

Meanwhile, the required information, including data such as the sender's quantum signatures and the positions of decoy states, is transmitted over a public channel without any prior encryption. While the protocol specifies that the sender delivers messages to the trusted center (TC), it lacks supplementary safeguards to ensure exclusive reception by the TC. Neither does the protocol guarantee that unauthorized third parties that intercept the data cannot extract meaningful valuable information. It is critical to note that although quantum key distribution (QKD) provides information-theoretic unconditional security, this security guarantee only applies to the key generation process, not to the encryption of quantum signature data during transmission.

Thereby, in order to judge which one of m and m' in public board is Alice's message, TC has no alternative but to recover the secure key from $U_{S_{AT}}|M\rangle$ or $U_{S_{AT}}|M'\rangle$. For the same reason, Alice isn't able to ensure the following two cases: (1) The receiver of the message that Alice tells to TC is indeed TC; (2) The message received by Alice really comes from TC. This explains why Eve can intercept the signature $U_{S_{AT}}|M\rangle$ during transmission without being detected, as the protocol provides no authentication of the communicating parties.

A critical vulnerability lies in the fact that the verifying unitary operator $U_V = 2|M\rangle\langle M| - I$ depends entirely on the message m , which is publicly announced on the board. Since m is known to all parties, anyone, including an eavesdropper, can construct U_V independently. This means that the verification procedure provides no authentication: the ability to verify a signature is not restricted to the intended recipient but is available to any adversary who reads the public board. Combined with the unprotected transmission of the signature $U_{S_{AT}}|M\rangle$, this allows Eve to recover the secret key K_{AT} by simply applying U_V to the intercepted signature.

TC is only able to confirm that the received message $U_{S_{AT}}|M'\rangle$ was signed using the shared key. So, Eve can prove to TC that he is Alice. Since TC can only verify that a message

was signed with K_{AT} , and Eve has recovered K_{AT} , Eve can successfully impersonate Alice to TC. This same vulnerability also exists in the TC-Bob phase. Similarly, the phase between TC and Bob also faces the same problems. As a result, TC's confirmation doesn't provide message authentication and non-repudiation. In addition, message integrity cannot be guaranteed by the signature key because Eve can obtain the secret key to sign any message. Therefore, none of the three basic conditions is satisfied in YKLY'2015 [4].

4. An AQS scheme on Teleportation-QOTP

In this section, we present an arbitrated quantum signature (AQS) scheme designed to avoid the weaknesses identified in the Grover-based design. We first define the system model and assumptions, then describe the protocol, and finally provide a formal security analysis.

4.1. System model and assumptions

The proposed scheme is designed for an arbitrated setting where two mutually distrusting parties require a trusted arbitrator to resolve disputes. The following model formalizes the participants, their trust relationships, the communication channels, the adversary capabilities, and the security properties that the protocol must satisfy.

4.1.1. Participants and main quantum primitives

The protocol involves three parties. Alice is the signer, who holds a quantum message $|p\rangle$ whose classical description is known to her, and wishes to generate a valid signature. Bob is the receiver and verifier, who authenticates the signed message with the assistance of the arbitrator. Trent is the arbitrator, who is trusted to facilitate signature generation, verification, and dispute resolution.

The quantum message to be signed, denoted by $|p\rangle$, is assumed to be a state whose classical description is known to Alice. Consequently, Alice can prepare multiple identical copies of $|p\rangle$ without violating the no-cloning theorem. This assumption is adopted throughout the protocol and distinguishes our scheme from signature protocols designed for completely unknown quantum states.

Our protocol employs three main quantum primitives: the strengthened quantum one-time pad for encrypting quantum states (see Section 2 for details), quantum teleportation for transferring quantum states (see Section 2 for details), and the swap test for comparing quantum states during the verification phase, which we now describe in more detail.

The swap test is a well-established probabilistic procedure for comparing unknown quantum states that has been widely adopted in quantum information processing. It takes two states $|\psi\rangle$ and $|\phi\rangle$ as input, and applies a controlled-SWAP gate between them, controlled on an auxiliary qubit initialized in $|+\rangle$, followed by a Hadamard gate on the auxiliary qubit and a measurement in the computational basis. When the two states are identical, the test always outputs "equal". When they differ, the test outputs "equal" with probability $(1 + F)/2$ and "not equal" with probability $(1 - F)/2$, where $F = |\langle\psi|\phi\rangle|^2$ denotes the fidelity between the two states. For orthogonal states ($F = 0$), this reduces to $1/2$ each. By repeating the test m times on independent copies, the probability that all m tests erroneously output "equal" for two different states is $[(1 + F)/2]^m$. For orthogonal states ($F = 0$), this reduces to $(1/2)^m = 2^{-m}$. The resource cost of a single swap test is modest: it requires one auxiliary qubit, one controlled-SWAP gate per pair of compared qubits, and one Hadamard gate, plus a single-qubit measurement. For single-qubit states, these operations are constant. For n -qubit registers, the number of controlled-SWAP gates scales as $O(n)$, while the auxiliary qubit and measurement overhead remain constant. When the test is repeated $m = \lambda$ times, the total resource cost scales linearly with λ . In

cryptographic protocols, it is standard practice to anchor all security-critical parameters to a single security parameter, typically denoted by λ , which quantifies the overall security strength of the system. The defining property is that any adversary should succeed with probability at most $2^{-\lambda}$, which is a negligible function in λ . Following this principle, we set the repetition count of the swap test to $m = \lambda$. This directly bounds the verification error probability by $2^{-\lambda}$, ensuring that the swap test achieves the same security level as the other cryptographic building blocks. The same parameter λ governs the length of pre-shared authentication keys and the number of bits consumed by the quantum one-time pad, so that every component of the protocol operates at a consistent security level. For practical deployment, the value of λ can be chosen according to the required security level. A choice of $\lambda = 128$ provides security against classical adversaries and corresponds to the widely adopted AES-128 standard. When resistance to quantum-speedup attacks is desired, for instance to account for Grover's algorithm, one may adopt $\lambda = 256$, which restores the effective security to 2^{128} against quantum search.

4.1.2. Trust model

Trent is trusted. The arbitrator follows the protocol honestly and does not collude with either Alice or Bob. Trent is responsible for key distribution, signature verification, and dispute resolution. This assumption reflects the role of trusted institutions such as notaries, certification authorities, and judicial bodies in conventional legal and commercial frameworks. In such settings, the trusted third party is assumed to be impartial and bound by procedural rules, which makes the assumption both natural and well-motivated. We further assume that Trent does not collude with either party individually. If Trent were to collude with Alice, he could help her repudiate a valid signature. If Trent were to collude with Bob, he could help him forge a signature. Both scenarios would render the arbitrated model meaningless and are therefore excluded from our security analysis.

Alice and Bob are mutually distrusting. Each party may attempt to cheat the other. Alice may try to repudiate a signature she has generated, while Bob may attempt to forge a signature or alter a signed message. This assumption models realistic adversarial behavior in contractual relationships, where neither party has reason to trust the other's honesty and each acts in their own interest.

No collusion between Alice and Bob. We assume that Alice and Bob do not collude jointly to defraud the arbitrator or to compromise the protocol. This is a standard and well-motivated assumption in arbitrated signature schemes. If Alice and Bob were willing to collude, they would be acting in concert rather than in conflict, and the protocol would no longer be needed. The purpose of an arbitrated signature is to protect each party against a potentially dishonest counterparty. When both parties choose to cooperate, there is simply no dispute for the arbitrator to resolve, and the scenario falls outside the intended scope of the model.

4.1.3. Channel assumptions

We assume that the classical channels between all parties are authenticated but not necessarily confidential. An adversary may observe the messages on these channels but cannot modify them without detection. The quantum channels are assumed to be fully insecure: an adversary may intercept, measure, or replace quantum states in transit. The security of our protocol does not rely on the confidentiality of the quantum channels, since all transmitted quantum states, with the exception of one plaintext copy used for local verification, are encrypted via the quantum one-time pad before transmission. We also assume that quantum key distribution links are available between each pair of parties to establish the pre-shared keys K_{AT} and K_{BT} .

4.1.4. Adversary model

We consider two classes of adversaries. An external adversary is not a legitimate participant in the protocol and does not possess any pre-shared keys, but has full access to the classical and quantum channels and may eavesdrop on all classical messages, intercept quantum states in transit, inject forged quantum states or classical messages into the channels, capture a transmitted state and perform arbitrary quantum operations on it before forwarding the result to the intended recipient, and record and later retransmit previously observed messages or states. The external adversary is assumed to be computationally unbounded but limited by the laws of quantum mechanics. Internal adversaries consist of a dishonest Alice or a dishonest Bob, either of whom may deviate arbitrarily from the protocol specification. A dishonest Alice may attempt to repudiate a valid signature that she has genuinely generated by tampering with the quantum states she prepares, sending inconsistent classical messages, or exploiting the dispute resolution procedure. A dishonest Bob may attempt to forge a valid signature by manipulating the verification process, altering the quantum states he receives, or presenting modified evidence to Trent during dispute resolution. Both internal adversaries have full knowledge of the protocol specification and access to their own keys and quantum resources, but they do not possess the keys held by the other party or by Trent.

4.1.5. Security properties and formal security games

Under the above model, the proposed scheme aims to satisfy unforgeability and non-repudiation. We now define these properties through formal security games between a challenger and an adversary.

Unforgeability. Unforgeability requires that no adversary can generate a valid signature attributable to Alice without Alice's participation. We define the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, where $\mathcal{A}$ may be an external adversary or a dishonest Bob. In the setup phase, $\mathcal{C}$ generates the pre-shared keys K_{AT} and K_{BT} and gives K_{BT} to $\mathcal{A}$. If $\mathcal{A}$ is an external adversary, it receives no keys. In the query phase, $\mathcal{A}$ may polynomially many times request Alice to sign messages of its choice. For each query with message $|p_i\rangle$, $\mathcal{C}$ runs the protocol on behalf of Alice and Trent and returns the resulting transcript to $\mathcal{A}$. In the forgery phase, $\mathcal{A}$ outputs a message $|p^*\rangle$ and a claimed signature σ^* , where $|p^*\rangle$ was never queried in the previous phase. In the verification phase, $\mathcal{C}$ runs the verification procedure on behalf of Trent. $\mathcal{A}$ wins if Trent outputs "valid" and attributes the signature to Alice. The scheme satisfies unforgeability if, for any polynomial-time adversary $\mathcal{A}$, the probability that $\mathcal{A}$ wins this game is a negligible function in λ , where λ is the security parameter.

Non-repudiation. Non-repudiation requires that if Alice genuinely generates a valid signature, she cannot later deny having done so. We define the following game between a challenger $\mathcal{C}$ and a dishonest Alice $\mathcal{A}$. In the setup phase, $\mathcal{C}$ generates the pre-shared keys K_{AT} and K_{BT} and gives K_{AT} to $\mathcal{A}$, and also initializes Bob and Trent with their respective keys. In the signature generation phase, $\mathcal{A}$ follows the protocol honestly and generates a valid signature σ on a message $|p\rangle$ of her choice, with Bob and Trent both following the specification honestly. In the repudiation attempt phase, $\mathcal{A}$ outputs a claim that the signature σ is invalid, together with any evidence she wishes to present to the arbitrator. In the dispute resolution phase, $\mathcal{C}$ invokes Trent's dispute resolution procedure using the evidence provided by $\mathcal{A}$ and any records held by Bob and Trent. $\mathcal{A}$ wins if Trent concludes that the signature is invalid. The scheme satisfies non-repudiation if, for any polynomial-time dishonest Alice $\mathcal{A}$, the probability that $\mathcal{A}$ wins this game is a negligible function in λ , where λ is the security parameter.

4.2. The proposed scheme

To satisfy the security properties defined in Section 4.1, the proposed scheme combines entanglement distribution, quantum teleportation [24], and strengthened quantum one-time pad encryption [16]. The protocol consists of two phases, the initializing phase and the signing-verifying phase. Using a single qubit as an example, the proposed scheme can be described as follows.

4.2.1. Initializing phase

In this phase, the secret keys are shared and the Bell states are generated.

(I1) Alice and Bob share their secret keys $K_{AT} \in \{0, 1\}^{16}$, $K_{BT} \in \{0, 1\}^{24}$ with Trent by QKD protocols [19–21] respectively, which are proved to be unconditionally secure [25,26]. K_{AT} is divided into three segments: K'_{AT} (bits 1–4), K''_{AT} (bits 5–8), and K'''_{AT} (bits 9–16). Similarly, K_{BT} is divided into four segments: K'_{BT} (bits 1–4), K''_{BT} (bits 5–12), K'''_{BT} (bits 13–16), and $K^{(4)}_{BT}$ (bits 17–24). In practice, QKD can operate continuously in the background to maintain a pool of pre-shared keys. The keys required for each signature are then drawn from this pool, so that the key generation does not delay the signing process.

(I2) Trent generates three Bell states $|\varphi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. He sends one particle of the first Bell state to Alice and the other to Bob, establishing the shared state $|\varphi^+\rangle_{AB}$, which will be used in Step (S2). He sends one particle of the second Bell state to Bob and keeps the other, establishing a shared Bell state for Step (S4). He sends one particle of the third Bell state to Bob and keeps the other, establishing a shared Bell state for Step (S6). We assume that these entanglement distribution steps can be realized via standard quantum communication techniques, using quantum repeaters [27] if the parties are far away from each other [25]. The security of the distributed entanglement can be ensured by entanglement purification and quantum state authentication [28].

4.2.2. Signing-verifying phase

In this phase, a quantum message $|p\rangle$ is signed by Alice and verified by Trent and Bob. The first three steps are performed by Alice.

(S1) Alice prepares three identical copies of the quantum message: $|p_1\rangle$, $|p_2\rangle$, and $|p_3\rangle$. As discussed in Section 4.1, this is possible because Alice knows the classical description of $|p\rangle$.

(S2) Alice combines message particle $|p_1\rangle$ with Bell state $|\varphi^+\rangle_{AB}$ to obtain a three-particle entangled state $|p_1\rangle_A \otimes |\varphi^+\rangle_{AB}$, and performs a Bell measurement on the message particle and the Bell state particle in her hand. Then, the three-particle entangled state $|p_1\rangle_A \otimes |\varphi^+\rangle_{AB}$ becomes the state

$$[I \otimes U(X_1)]|\varphi^+\rangle_{AA} \otimes U(X_1)|p_1\rangle_B. \quad (6)$$

That is, the state of the corresponding Bell state particle in Bob's hand becomes $U(X_1)|p_1\rangle$. Here, X_1 is determined by the state $[I \otimes U(X_1)]|\varphi^+\rangle$ of the two particles in Alice's hand. Because the state of the two particles in Alice's hand randomly collapses into one of the four Bell states, X_1 is randomly chosen from the set $\{00, 01, 10, 11\}$. Note that Alice can obtain X_1 .

(S3) Alice sends $|p_2\rangle$ and $E_{K'_{AT}}|p_3\rangle$ to Bob and Trent, respectively. After encrypting the second particle of the Bell state in her hand by $E_{K''_{AT}}$ and $|X_1\rangle$ by K'''_{AT} , she sends the particle pair (its state is $[I \otimes E_{K''_{AT}} U(X_1)]|\varphi^+\rangle$) and $E_{K'''_{AT}}|X_1\rangle$ to Trent.

Next step is performed by Bob.

(S4) Bob combines message particle $U(X_1)|p_1\rangle$ with the Bell state shared with Trent (established in Step (I2) for this purpose) to obtain a three-particle entangled state, and

performs a Bell measurement on the message particle and the Bell state particle in his hand. The three-particle entangled state then becomes

$$[I \otimes U(X_2)]|\varphi^+\rangle_{BB} \otimes U(X_2)U(X_1)|p_1\rangle_T. \quad (7)$$

Then, the state of the corresponding Bell state particle in Trent's hand becomes $U(X_2)U(X_1)|p_1\rangle_T$. Here, X_2 is determined by the state $[I \otimes U(X_2)]|\varphi^+\rangle$ of the two particles in Bob's hand. Note that Bob can obtain X_2 . After encrypting the second particle of the Bell state in his hand by $E_{K'_{BT}}$ and $|X_2\rangle$ by K''_{BT} , Bob sends the particle pair (its state is $[I \otimes E_{K'_{BT}} U(X_2)]|\varphi^+\rangle$) and $E_{K''_{BT}}|X_2\rangle$ to Trent.

The next two steps are performed by Trent.

(S5) Trent decrypts the particle pair from Alice using K''_{AT} and measures it to obtain X_1 . He then decrypts $E_{K'''_{AT}}|X_1\rangle$ using K'''_{AT} to obtain $|X_1\rangle$, and compares the measured X_1 with the decrypted $|X_1\rangle$. If they differ, the protocol aborts. If they match, he decrypts the particle pair from Bob using K'_{BT} and measures it to obtain X_2 . He then decrypts $E_{K''_{BT}}|X_2\rangle$ using K''_{BT} to obtain $|X_2\rangle$, and compares the measured X_2 with the decrypted $|X_2\rangle$. If they differ, the protocol aborts. Trent then applies $U(X_1)$ and $U(X_2)$ to recover $|p_1\rangle$ from $U(X_2)U(X_1)|p_1\rangle$. He compares the recovered $|p_1\rangle$ with $|p_3\rangle$ using the swap test described in Section 4.1. If they differ, the protocol aborts. Otherwise, he proceeds to the next step.

(S6) Trent combines the recovered message particle $|p_1\rangle$ with the Bell state shared with Bob (established in Step (I2) for this purpose) to obtain a three-particle entangled state, and performs a Bell measurement on the message particle and the Bell state particle in his hand. The three-particle entangled state then becomes

$$[I \otimes U(X_3)]|\varphi^+\rangle_{TT} \otimes U(X_3)|p_1\rangle_B. \quad (8)$$

Consequently, the state of the corresponding Bell state particle in Bob's hand becomes $U(X_3)|p_1\rangle$. After encrypting the second particle of the Bell state in his hand by $E_{K'''_{BT}}$ and $|X_3\rangle$ by $K^{(4)}_{BT}$, Trent sends the particle pair (its state is $[I \otimes E_{K'''_{BT}} U(X_3)]|\varphi^+\rangle$) and $E_{K^{(4)}_{BT}}|X_3\rangle$ to Bob.

The last step is performed by Bob to verify and accept message.

(S7) Bob decrypts the particle pair from Trent using K'''_{BT} and measures it to obtain X_3 . He then compares this value with $|X_3\rangle$ obtained by decrypting $E_{K^{(4)}_{BT}}|X_3\rangle$ using $K^{(4)}_{BT}$. If they differ, the protocol aborts. Otherwise, Bob applies $U(X_3)$ to the state $U(X_3)|p_1\rangle$ obtained from the teleportation, thereby recovering $|p_1\rangle$. He then compares the recovered $|p_1\rangle$ with $|p_2\rangle$ using the swap test described in Section 4.1. If they match, Bob accepts the message. Otherwise, the protocol aborts.

In summary, Alice transmits the quantum message to Trent via two rounds of teleportation through Bob. Trent verifies the message and teleports it back to Bob, who performs a final verification before acceptance. Throughout the protocol, all quantum states are protected by the strengthened quantum one-time pad, and all state comparisons are performed using the swap test. For clarity, the protocol execution flow is illustrated in Figure 3.

4.3. Security analysis of the proposed scheme

We now analyze the security of the proposed scheme with respect to the formal definitions of non-repudiation and unforgeability established in Section 4.1. The analysis considers the three classes of adversaries defined in our model: a dishonest Alice, a dishonest Bob, and external adversaries. We then discuss the resilience of the protocol against other standard attack vectors.

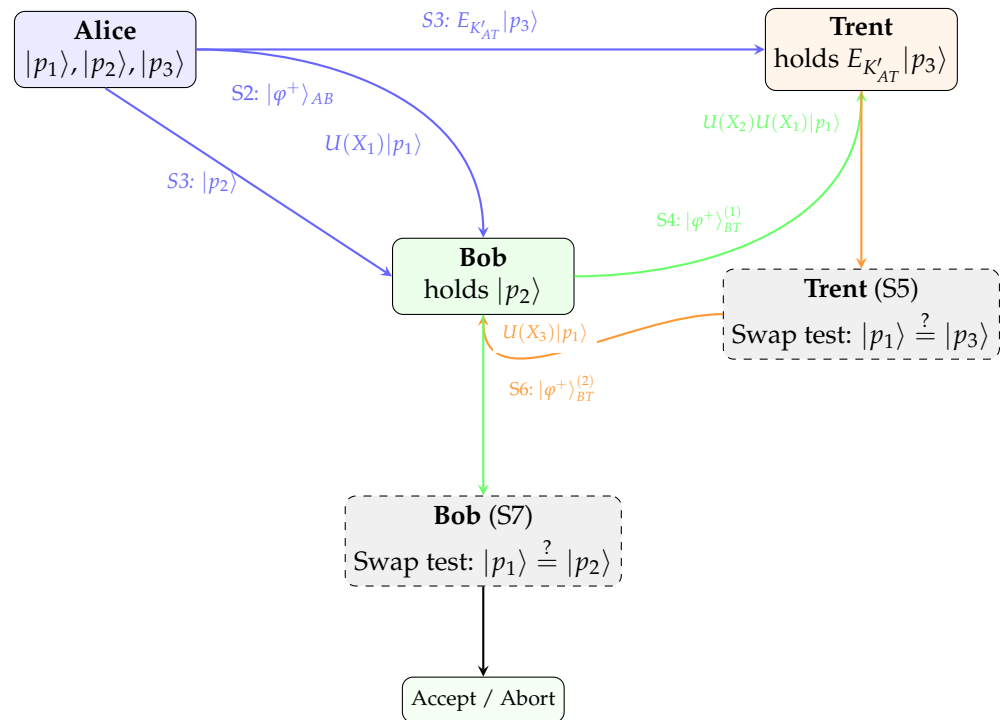


Figure 3. Flow of the three message copies and Bell state consumption in the proposed scheme.

4.3.1. Non-repudiation

We show that the proposed scheme satisfies the non-repudiation game defined in Section 4.1. Let $\mathcal{A}$ be a dishonest Alice who attempts to repudiate a valid signature.

In the signature generation phase of the game, Alice honestly produces a valid signature on a message $|p\rangle$ of her choice. The protocol executes normally with Bob and Trent following the specification. In the repudiation attempt phase, Alice outputs a claim that the signature is invalid and presents evidence to the arbitrator. For Alice to win, Trent must conclude that the signature is invalid despite Alice having genuinely generated it.

The analysis below considers a stronger adversary than the one defined in the non-repudiation game. Alice is allowed to deviate already during the signature generation phase, not only during the repudiation attempt phase. If the protocol resists this stronger adversary, it certainly resists the standard one. We now show that any attempt by Alice to repudiate is detected by the protocol's verification mechanisms. Suppose Alice mounts the strongest possible attack by deviating from the protocol in every step where she has control. Specifically, in Step (S3), Alice sends $|p'_2\rangle$ to Bob, $E_{K'_{AT}}|p'_3\rangle$ to Trent, and a particle pair encoding X'_1 together with $E_{K'_{AT}}|X'_1\rangle$ to Trent. These may differ arbitrarily from the honest values.

For the protocol to proceed beyond Step (S5), Trent must successfully verify both X_1 and X_2 , and the swap test between the recovered $|p_1\rangle$ and $|p_3\rangle$ must pass. The verification of X_1 requires that the value measured from Alice's particle pair matches the decrypted $|X'_1\rangle$. Since Alice controls both the particle pair and the encrypted X'_1 , she can ensure this check passes by sending consistent values. However, the swap test imposes a stronger constraint. Trent recovers the teleported state and compares it with $|p'_3\rangle$. Let $|p_1^{\text{rec}}\rangle$ denote the state recovered by Trent. For the swap test to pass with high probability, we must have $|p_1^{\text{rec}}\rangle \cong |p'_3\rangle$. Given the teleportation chain and assuming Bob behaves honestly, Trent's recovery yields $|p_1^{\text{rec}}\rangle = U(X'_1)U(X_1)|p_1\rangle$ (up to a global phase arising from Pauli commutation), which gives the condition $U(X'_1)U(X_1)|p_1\rangle \cong |p'_3\rangle$.

Now suppose the protocol continues to Step (S7). Bob receives the state teleported by Trent, applies $U(X_3)$ to recover $U(X'_1)U(X_1)|p_1\rangle$, and performs a swap test with $|p'_2\rangle$. For Bob to accept, the swap test must pass, which implies $|p'_2\rangle \cong U(X'_1)U(X_1)|p_1\rangle \cong |p'_3\rangle$.

This is the crucial observation: if Bob accepts the signature, then the message $|p'_2\rangle$ held by Bob, the message $|p'_3\rangle$ held by Trent, and Alice's original $|p_1\rangle$ are provably connected through the relation $|p'_2\rangle \cong |p'_3\rangle \cong U(X'_1)U(X_1)|p_1\rangle$. Trent, who possesses both $|p'_3\rangle$ and the records of X'_1 and X_1 , can verify this chain of derivation. Since $|p'_2\rangle$ is the message that Bob received from Alice, and Trent can confirm that it originated from Alice's original $|p_1\rangle$, Alice cannot later claim that she did not sign this message.

Even in the extreme case where Alice sends $|p'_2\rangle$ that differs from $|p_1\rangle$, the consistency enforced by the protocol ensures that $|p'_2\rangle \cong |p'_3\rangle$, meaning that whatever message Bob received is exactly what Trent verified. Alice cannot later claim that Bob received a different message than what she sent, because Trent's records provide an unforgeable audit trail.

Therefore, a dishonest Alice cannot win the non-repudiation game with non-negligible probability.

4.3.2. Unforgeability

We show that the proposed scheme satisfies the unforgeability game defined in Section 4.1. Let $\mathcal{A}$ be a polynomial-time adversary, which may be either a dishonest Bob or an external adversary. We consider each case separately.

Dishonest Bob. A dishonest Bob possesses the key K_{BT} and participates in the protocol as the designated receiver. His goal is to forge a valid signature—that is, to produce a message-signature pair that passes Trent's verification and is attributed to Alice, without Alice having signed that message.

Bob's capabilities in the protocol are as follows. After Step (S3), Bob holds $|p_2\rangle$ (the plaintext copy sent by Alice) and $U(X_1)|p_1\rangle$ (the teleported state). In Step (S4), Bob performs a Bell measurement and learns X_2 . After Step (S7), Bob additionally holds $U(X_3)|p_1\rangle$ (the state teleported back by Trent) and learns X_3 .

To forge a signature, Bob must produce a message $|p^*\rangle$ and evidence that causes Trent to attribute it to Alice. There are two stages at which Bob might attempt to cheat.

In the first stage (during protocol execution), Bob could modify $|p_2\rangle$ or apply a non-identity operation G on $U(X_1)|p_1\rangle$ before teleporting it to Trent in Step (S4). If Bob applies G , the state reaching Trent in Step (S5) becomes $U(X_2)GU(X_1)|p_1\rangle$. For the protocol to proceed, Trent must successfully verify X_1 and X_2 . The verification of X_1 depends on Alice's encryption with K_{AT} , which Bob cannot forge because he does not possess K_{AT} . If Bob sends an inconsistent X_2 , the decryption check in Step (S5) fails and the protocol aborts. If Bob passes the X_2 check but modified the quantum state, Trent applies $U(X_1)$ and $U(X_2)$ to the received state. Since Pauli operators either commute or anti-commute, the recovered state becomes $U(X_1)GU(X_1)|p_1\rangle$ (up to a global phase). Trent then compares it with $|p_3\rangle$ via the swap test. Let $F_G = |\langle p_1|U(X_1)GU(X_1)|p_1\rangle|^2$ denote the fidelity between the recovered state and the reference state $|p_3\rangle$. The swap test detects the mismatch with probability $1 - [(1 + F_G)/2]^\lambda$. When $G = I$, we have $F_G = 1$ and the test always accepts. When $G \neq I$, the detection probability depends on F_G ; in the worst case for the protocol, G may be chosen such that $F_G \approx 1$ (e.g., a near-identity operation that preserves the message up to a small perturbation), yielding a low detection probability. The protocol therefore relies on the fact that Bob cannot make F_G arbitrarily close to 1 without knowing $|p_1\rangle$, which is protected by the encryption and teleportation chain.

In the second stage (after protocol completion), Bob possesses $|p_2\rangle$ and $U(X_3)|p_1\rangle$. After applying $U(X_3)$, Bob recovers $|p_1\rangle$ and compares it with $|p_2\rangle$. At this point, Bob already holds the valid message. To forge a different message, Bob would need to produce a

new $|p^*\rangle$ and corresponding evidence that Trent accepts as originating from Alice. However, Trent retains the encrypted record $E_{K'_{AT}}|p_3\rangle$ and the particle pairs from both Alice and Bob. Any forgery attempt by Bob would require him to produce a state consistent with Trent's records, which are protected by K_{AT} that Bob does not possess.

Bob's knowledge of X_2 and X_3 does not assist in forgery, because these values are random Bell measurement outcomes that reveal no information about the signed message. They are independent of the message content and are used solely for reconstructing the teleported state.

Therefore, a dishonest Bob cannot win the unforgeability game with non-negligible probability.

External adversary. An external adversary does not possess any of the pre-shared keys K_{AT} or K_{BT} . With the exception of the plaintext copy $|p_2\rangle$ sent to Bob in Step (S3), all quantum states transmitted during the protocol are encrypted via the strengthened quantum one-time pad, which provides information-theoretic security when the encryption keys are uniformly random and used only once. Specifically, in Step (S3), Alice encrypts $|p_3\rangle$ with K'_{AT} and the Bell state particle pair with K''_{AT} . In Step (S4), Bob encrypts his particle pair with K'_{BT} . In Step (S6), Trent encrypts his particle pair with K'''_{BT} . Without access to these keys, the adversary cannot decrypt the ciphertext to recover the underlying quantum states, nor can it extract meaningful information via the attacks discussed in Section 4.3.3. Consequently, the adversary cannot produce a ciphertext that would pass Trent's verification. Furthermore, any active interference by the adversary, such as intercepting and replacing a quantum state in transit, introduces detectable discrepancies. These discrepancies manifest in two ways. First, the decrypted values X_1 , X_2 , and X_3 obtained by Trent and Bob will not match the encrypted values sent by the legitimate parties, causing the protocol to abort in Steps (S5) and (S7). Second, even if the adversary somehow managed to preserve the classical parameters, the swap test comparisons in Steps (S5) and (S7) would detect the mismatch between the recovered $|p_1\rangle$ and the stored copies $|p_3\rangle$ and $|p_2\rangle$, respectively. The swap test fails to detect a difference with probability $[(1 + F)/2]^\lambda$, where F is the fidelity between the recovered state and the reference state. For orthogonal states ($F = 0$), this reduces to $2^{-\lambda}$, which is negligible. Hence, an external adversary cannot win the unforgeability game with non-negligible probability.

4.3.3. Security against other attacks

We now discuss the resilience of the proposed scheme against several additional types of attacks that are relevant to quantum signature protocols.

Replay attacks. A replay attack against the proposed scheme would consist of recording the quantum states and classical messages transmitted during a legitimate execution and later retransmitting them in a new session. Two protocol features prevent this. First, every execution uses freshly generated QKD keys K_{AT} and K_{BT} . The replayed states, encrypted under an old key, would not decrypt correctly under the current key held by Trent or Bob, causing the verification in Steps (S5) or (S7) to fail. Second, the Bell measurement outcomes X_1 , X_2 , and X_3 are uniformly random in each execution. Even if a replayed quantum state were somehow accepted, the accompanying classical X_i values from the previous session would not match the values independently measured by Trent and Bob in the current session, and the comparisons in Steps (S5) and (S7) would abort the protocol. Replay attacks are therefore effectively prevented under the assumptions of fresh keys and unique per-execution Bell outcomes.

Intercept-and-resend attacks. In the proposed scheme, an adversary who intercepts a quantum transmission, for instance the particle pair sent by Alice to Trent in Step (S3), faces two obstacles. First, the state is encrypted via the strengthened QOTP using K''_{AT} , which the

adversary does not possess. The QOTP renders the ciphertext statistically indistinguishable from a maximally mixed state, so any measurement yields no information about the underlying plaintext. Second, if the adversary replaces the intercepted state with an arbitrary state and forwards it to Trent, the decryption in Step (S5) will produce a value that does not match the separately transmitted encrypted record $E_{K_{AT}'''}|X_1\rangle$, causing the protocol to abort. The same argument applies to the particle pairs transmitted in Steps (S4) and (S6). If the adversary instead targets the plaintext copies $|p_2\rangle$ or $|p_3\rangle$, any modification will be detected by the swap test comparisons in Steps (S5) and (S7). We note that QOTP provides confidentiality and plaintext hiding; integrity and authentication are enforced by these additional protocol checks. The adversary therefore gains no advantage from intercept-and-resend attacks.

Man-in-the-middle attacks. A man-in-the-middle adversary must simultaneously impersonate one party to another on both directions of a communication channel. In the proposed scheme, this requires overcoming three independent defenses. First, the classical channels are authenticated, so any modification of the encrypted X_i values or other classical messages is detected. Second, all quantum states are QOTP-encrypted; without the corresponding key, the adversary can neither extract the plaintext nor re-encrypt a modified state to pass verification. Third, even if the adversary successfully impersonates one party on one leg, the cross-verification performed by Trent in Step (S5) compares the independently received X_1 and X_2 values from Alice and Bob. A discrepancy introduced on any single channel, for example by replacing the particle pair and encrypted X_1 sent by Alice to Trent in Step (S3), will be detected when Trent decrypts and finds mismatching values. The adversary cannot produce a valid encryption of a forged X_1 without K_{AT} . Symmetric arguments apply to the other communication legs. Under the assumptions of authenticated classical channels and confidential pre-shared keys, man-in-the-middle attacks are effectively prevented.

Entanglement attacks. An adversary may attempt to entangle an ancilla with a transmitted quantum state and later measure the ancilla to extract information. In the proposed scheme, all quantum states transmitted over the insecure channels are QOTP-encrypted. The ciphertext appears maximally mixed to any party without the decryption key. If the adversary entangles an ancilla with the ciphertext, the reduced state of the ancilla remains independent of the plaintext. Formally, for a QOTP-encrypted state $\frac{1}{2^n} \sum_{a,b} X^a Z^b \rho Z^b X^a \otimes |a,b\rangle\langle a,b|$, tracing out the ciphertext leaves the ancilla uncorrelated with the original message ρ . The adversary therefore gains no information, and this protection holds against computationally unbounded adversaries due to the information-theoretic security of the QOTP.

Collective and coherent attacks. In collective attacks, the adversary processes each quantum signal individually but stores ancillae for joint measurement across multiple protocol runs. In coherent attacks, all signals may be processed jointly. The proposed scheme resists these attacks in the sense of preserving plaintext confidentiality across executions. Each execution uses independent, freshly generated QKD keys, so the encrypted states from different runs are statistically independent. Joint measurement across multiple ciphertexts yields no more information than individual analysis. Within a single execution, the security follows from the standard QOTP confidentiality guarantee, which holds against unbounded adversaries. The keys themselves are generated by QKD and are information-theoretically secure.

As above, we have shown that the proposed scheme satisfies non-repudiation and unforgeability under the formal adversarial model defined in Section 4.1. The analysis considers dishonest internal parties and external adversaries separately, and shows that neither class can succeed with non-negligible probability. Together, QOTP encryption,

authenticated classical channels, cross-verification by Trent, and per-execution fresh keys protect the protocol against replay, intercept-and-resend, man-in-the-middle, entanglement, and collective or coherent attacks.

5. Resource overhead and simulation

This subsection first analyzes resource overheads of the proposed scheme and YKLY'2015, and then simulates the attack to the proposed scheme.

5.1. Resource overhead analysis of the proposed scheme and YKLY'2015

To assess the practical cost of the security guarantees established in Section 4.3, we now analyze the resource consumption of the proposed scheme. For a single quantum message, all core operations of the proposed scheme maintain constant-level computational complexity. In this subsection, we divide the protocol execution into the initialization stage (I1-I2) and the signal interaction stage (S1-S7) to conduct a comprehensive resource overhead analysis, covering key consumption, qubit overhead, quantum communication overhead, classical communication overhead, and computational complexity of each step.

Step (I1): This step completes QKD key generation and distribution, with a total key consumption of 40 bits per protocol execution. The key K_{AT} (16 bits) is divided into three segments: K'_{AT} (4 bits) for encrypting $|p_3\rangle$, K''_{AT} (4 bits) for encrypting the Bell state particle pair, and K'''_{AT} (8 bits) for encrypting X_1 . The key K_{BT} (24 bits) is divided into four segments: K'_{BT} (4 bits) for encrypting the Bell state particle pair, K''_{BT} (8 bits) for encrypting X_2 , K'''_{BT} (4 bits) for encrypting the Bell state particle pair in the verification phase, and $K^{(4)}_{BT}$ (8 bits) for encrypting X_3 . Each segment is used exactly once and is never reused. All keys are freshly generated via QKD for each protocol execution. The key generation involves only fixed quantum operations, with a computational complexity of $O(1)$.

Step (I2): No additional secret keys are consumed in this step. Three Bell states are prepared and serve as persistent entanglement resources, occupying 6 static qubits. A total of 4 qubits are transmitted from Trent to Alice and Bob (one particle to Alice and three particles to Bob, while Trent retains two particles locally), with no classical communication overhead. The Bell state preparation is realized by Hadamard and CNOT quantum gates, and the computational complexity is $O(1)$. These three Bell states must be preserved until their respective teleportation steps (S2, S4, and S6). Let t_{comm} denote the one-way classical communication latency between any two parties, and let t_{op} denote the time for local quantum operations and measurements. The total protocol execution time, which determines the required storage duration, is $t_{\text{total}} = 3t_{\text{comm}} + O(t_{\text{op}})$, where the factor of 3 accounts for the three rounds of teleportation. For metropolitan-scale distances on the order of tens of kilometers, t_{comm} is on the order of tens of microseconds in optical fiber, yielding t_{total} on the order of milliseconds. This is well within the coherence times demonstrated by current quantum memory technologies. For longer-distance applications, the required storage time increases accordingly, and practical deployment would require continued progress in coherence-preserving technologies.

Step (S1): No key resources are consumed. 3 dynamic qubits are occupied for quantum message storage, and there is no quantum and classical communication transmission overhead. The operation only includes single-qubit state preparation, with a computational complexity of $O(1)$.

Step (S2): This step consumes no keys and introduces no additional qubits. No quantum qubits and classical bits are transmitted. The core operation is Bell basis projective measurement, and the computational complexity is $O(1)$.

Step (S3): Partial segments of K_{AT} are consumed, including 4-bit K'_{AT} , 4-bit K''_{AT} , and 8-bit K'''_{AT} . No new persistent qubits are added. A total of 4 qubits are transmitted,

including quantum state $|p_2\rangle$, QOTP encrypted quantum state $E_{K'_{AT}}|p_3\rangle$, and Bell particle pair. Meanwhile, 2-bit encrypted parameter X_1 is transmitted via classical channel. The step combines QOTP encryption and Bell particle encryption operations, with an overall computational complexity of $O(1)$.

Step (S4): Partial segments of K_{BT} are consumed, including 4-bit K'_{BT} and 8-bit K''_{BT} . No new persistent qubits are occupied. A total of 2 encrypted Bell particle qubits are transmitted, accompanied by 2-bit classical encrypted parameter X_2 . The core operations are Bell measurement and QOTP encryption, and the overall computational complexity is $O(1)$.

Step (S5): No new key segments and persistent qubits are consumed. There is no quantum and classical communication overhead. The main operations are quantum state recovery and state comparison, with a computational complexity of $O(1)$.

Step (S6): It consumes partial segments of K_{BT} , including 4-bit K'''_{BT} and 8-bit $K^{(4)}_{BT}$. No extra persistent qubits are introduced. A total of 2 encrypted Bell particle qubits and 2-bit classical encrypted parameter X_3 are transmitted. The step integrates Bell measurement and QOTP encryption, with an overall computational complexity of $O(1)$.

Step (S7): No key resources and persistent qubits are consumed, with zero communication overhead of quantum and classical channels. The core operations are Pauli correction and quantum state comparison, and the computational complexity is $O(1)$.

As above, the stepwise resource overhead summary of the proposed scheme is presented in Table 2. By the same approach, we derive the resource overhead of the Grover-based scheme (i.e., YKLY'2015), as listed in Table 3. The proposed scheme differs architec-

Table 2. Stepwise resource overhead summary of the proposed scheme

Index	I1	I2	S1	S2	S3	S4	S5	S6	S7	Total
Key Consumed (bit)	40	0	0	0	0	0	0	0	0	40
Static and Dynamic Qubits	0	6	3	0	0	0	0	0	0	9
Quantum Transmit (qubit)	0	4	0	0	4	2	0	2	0	12
Classical Transmit (bit)	40	0	0	0	2	2	0	2	0	46
Computational Complexity	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(\lambda)$	$O(1)$	$O(\lambda)$	$O(1)$

¹ The 40 secret key bits are freshly generated via QKD for each protocol execution. Each key segment is used exactly once and is never reused.

² Maximum number of simultaneously occupied valid qubits throughout a complete protocol execution cycle (values are non-cumulative).

³ Quantum/Classical Transmission: Cumulative qubits/classical bits transmitted over all protocol stages, including communication for QKD initialization.

⁴ Computational Complexity: Asymptotic complexity for all quantum and classical operations within each step.

turally from YKLY'2015 in several respects. The proposed scheme introduces persistent static qubits and consumes more key material (40 bits per execution vs. 4 bits in YKLY'2015). These additional costs are incurred in exchange for stronger security guarantees. Unlike YKLY'2015, which is vulnerable to man-in-the-middle attacks that enable Alice's disavowal and Bob's forgery, the proposed scheme achieves unforgeability and non-repudiation through the combined use of strengthened QOTP encryption and arbitrated teleportation-based verification. The higher key consumption reflects the additional encryption and authentication steps required to protect against these attacks. Furthermore, quantum teleportation is adopted to reuse quantum channels, reducing the need for repeated quantum state transmissions.

The above analysis considers a single-qubit message. For a general n -qubit message $|p\rangle$, the resource requirements scale as follows.

Table 3. Stepwise resource overhead summary of YKLY'2015

Index	AT Total (AT1-AT8)	TB Total (TB1-TB8)	Overall Total
Key Consumed (bit)	2	2	4
Dynamic Qubits Used	8	8	8
Quantum Transmit (qubit)	8	8	16
Classical Transmit (bit)	13	13	26
Computational Complexity	$O(1)$	$O(1)$	$O(1)$

¹ Key Consumption: New secret key bits generated in each stage; the total row accumulates all independent QKD keys K_{AT} and K_{TB} .

² Dynamic Qubit Consumption: Temporary qubits occupied within each step (values are non-cumulative).

³ Quantum/Classical Transmission: Cumulative transmitted qubits and classical bits within each step.

⁴ Computational Complexity: Asymptotic complexity of all quantum and classical operations within each step.

Key consumption. Each qubit of the message requires independent encryption via the strengthened QOTP, which consumes a constant number of key bits per qubit. Specifically, for an n -qubit message, Alice must encrypt three copies of the message ($|p_1\rangle, |p_2\rangle, |p_3\rangle$) and the Bell state particle pairs used in teleportation. The X_i parameters each become $2n$ -bit classical values for an n -qubit message, and their encryption cost therefore scales as $O(n)$. The total key consumption therefore scales as $O(n)$, with a dominant contribution coming from the QOTP encryption of the n -qubit message copies.

Static qubits. Each teleportation of an n -qubit message consumes n Bell states. The protocol involves three rounds of teleportation, requiring $3n$ Bell states in total. Each Bell state occupies 2 qubits, yielding $6n$ static qubits. The total number of static qubits therefore scales as $O(n)$.

Dynamic qubits. Alice prepares three identical copies of the n -qubit message, occupying $3n$ dynamic qubits. During teleportation, each message qubit temporarily interacts with one half of a Bell state, but no additional persistent storage is required beyond the message copies themselves. The total dynamic qubit requirement is therefore $O(n)$.

Quantum communication. In Step (S3), Alice transmits $|p_2\rangle$ (n qubits) to Bob and $E_{K'_{AT}}|p_3\rangle$ (n encrypted qubits) to Trent. For an n -qubit message with qubit-wise teleportation, the Bell state particle pairs transmitted in Steps (S3), (S4), and (S6) each scale linearly with n , contributing $O(n)$ to the quantum communication overhead. The total quantum communication overhead is therefore $O(n)$, dominated by the transmission of the n -qubit message copies.

Classical communication. The QKD initialization step (I1) requires $O(n)$ classical bits for key distribution, as the total key length scales with n . For an n -qubit message with qubit-wise teleportation, each Bell measurement produces $2n$ classical bits. The encrypted records of X_1 , X_2 , and X_3 therefore scale as $O(n)$, contributing to the total classical communication overhead. The total classical communication overhead is therefore $O(n)$.

Computational complexity. The dominant operations are Bell measurements, QOTP encryption and decryption, and swap tests. For an n -qubit message, Bell measurements are performed qubit-wise on each of the three teleportation rounds, requiring $O(n)$ operations. The QOTP encryption applies Pauli gates independently to each qubit, also scaling as $O(n)$. Each swap test compares n -qubit registers using n controlled-SWAP gates, and the test is repeated λ times per comparison. Two comparisons are performed (in Steps S5 and S7), yielding $O(\lambda n)$ overall. The total computational complexity is therefore $O(\lambda n)$.

In summary, for an n -qubit message, key consumption, qubit overhead, and communication requirements scale as $O(n)$, while the computational complexity of repeated register

comparison scales as $O(\lambda n)$. This scaling remains linear in the message length n , which is a natural consequence of processing each qubit individually. Table 4 summarizes the asymptotic resource scaling.

Table 4. Asymptotic resource scaling for an n -qubit message.

Resource	Scaling
Key consumption	$O(n)$
Static qubits	$O(n)$
Dynamic qubits	$O(n)$
Quantum communication	$O(n)$
Classical communication	$O(n)$
Computational complexity	$O(\lambda n)$

5.2. Attack simulations of the proposed scheme

To complement the formal security analysis of Section 4.3, we now simulate the strongest attacks that a dishonest Alice or Bob can mount against the proposed scheme. By contrast, the vulnerabilities of YKLY'2015 follow directly from its protocol structure and require no simulation to verify.

5.2.1. Alice's attack simulation

For Alice's repudiation, we construct the strongest possible attack, in which Alice deviates maximally in every step under her control.

Alice's attack description. In Step (S3), Alice deviates from the protocol by sending a potentially forged state $|p'_2\rangle$ to Bob and $E_{K'_{AT}}|p'_3\rangle$ to Trent, where $|p'_2\rangle$ and $|p'_3\rangle$ may differ from the honest copies. Simultaneously, she sends a forged particle pair with state $[I \otimes E_{K'_{AT}} U(X'_1)]|\varphi^+\rangle$ and $E_{K'_{AT}}|X'_1\rangle$ to Trent, where X'_1 may differ from the true X_1 .

Verification phase Analysis. In Step (S5), Trent decrypts and verifies the received particles. We analyze three possible scenarios depending on Bob's behavior, since Alice does not control whether Bob is honest.

Scenario 1: Bob is honest. Bob follows the protocol ($G = I$, $X'_2 = X_2$). Trent measures X'_1 from Alice's particle pair and compares it with the decrypted $|X'_1\rangle$. If they match, Trent applies $U(X'_1)$ and $U(X_2)$ to the state $U(X_2)U(X_1)|p_1\rangle$. The recovered state is $|p_1^{\text{rec}}\rangle = U(X'_1)U(X_1)|p_1\rangle$ (up to a global phase). The swap test between $|p_1^{\text{rec}}\rangle$ and $|p'_3\rangle$ yields the condition

$$|p_1^{\text{rec}}\rangle \cong |p'_3\rangle \cong U(X'_1)U(X_1)|p_1\rangle. \quad (9)$$

This corresponds to the stronger malicious-signing adversary considered in the non-repudiation analysis of Section 4.3, where Alice is allowed to deviate during signature generation.

Scenario 2: Bob is dishonest, but his attack prevents the protocol from proceeding. Bob applies $G \neq I$ or sends $X'_2 \neq X_2$. If $U(X'_2)U(X_2)G \neq I$, the fidelity F between the state recovered by Trent and the reference state $|p_3\rangle$ satisfies $F < 1$, and the swap test detects the mismatch with probability $1 - [(1 + F)/2]^\lambda$. Unless Bob's deviation is chosen such that $F \approx 1$, this probability is close to 1, causing the protocol to abort. In this case, Bob has not accepted any signature, so Alice has nothing to repudiate. Her attack fails trivially.

Scenario 3: Bob is dishonest, but his attack inadvertently assists Alice. Bob applies $G \neq I$ and/or sends $X'_2 \neq X_2$, but the combination happens to satisfy $U(X'_2)U(X_2)G = I$. In this case, the swap test passes and the protocol proceeds despite Bob's deviation. Table 5 shows the required G for each pair (X'_2, X_2) . For a given pair (X'_2, X_2) , Bob's deviation G is chosen randomly from $\{I, \sigma_x, \sigma_y, \sigma_z\}$, and exactly one of these four choices satisfies

$U(X'_2)U(X_2)G = I$. Hence, if Bob chooses G uniformly at random, the probability that his deviation inadvertently allows the protocol to proceed is $1/4$. Even in this scenario, once the protocol proceeds to Step (S7) and Bob accepts, the traceability argument of Scenario 1 applies, and Trent can still link the accepted message to Alice.

Table 5. Required operation G for the protocol to proceed when Bob deviates by sending X'_2 instead of the true X_2 . The diagonal entries ($X'_2 = X_2$) correspond to $G = I$, i.e., Bob does not apply any additional operation.

$X'_2 \backslash X_2$	00	01	10	11
00	I	σ_x	σ_z	σ_y
01	σ_x	I	σ_y	σ_z
10	σ_z	σ_y	I	σ_x
11	σ_y	σ_z	σ_x	I

In practical systems, non-attack factors such as channel noise, device imperfections, and environmental disturbances introduce an additional baseline error rate, denoted by α . In Scenario 1 (Bob honest), the protocol proceeds deterministically from Alice's perspective, and the only source of error is the finite failure probability $[(1 + F)/2]^\lambda$ of the swap test, which reduces to $2^{-\lambda}$ for orthogonal states. In Scenario 3 (Bob inadvertently assists Alice), the protocol proceeds with ideal probability $1/4$ as shown above. Taking the worst-case view that Scenario 3 occurs, the total probability that Trent observes an erroneous acceptance is modeled as $p = \frac{1}{4} + \alpha$, where the $1/4$ term accounts for the probability that Bob's random deviation accidentally satisfies $U(X'_2)U(X_2)G = I$, and α is the baseline error rate due to physical noise, treated as an independent additive contribution in this phenomenological model. A detection threshold β is set: the transmission is accepted if the observed bit-error rate $\hat{\varepsilon} \leq \beta$, and rejected otherwise. The probability that Alice's attack succeeds (remains undetected) is $P_{\text{success}}(n) = \Pr(\hat{\varepsilon} \leq \beta \mid p = \frac{1}{4} + \alpha)$. For n transmitted quantum bits, let the observed number of erroneous bits be k . Then k follows a binomial distribution $k \sim \text{Binomial}(n, p)$, and the observed bit-error rate is $\hat{\varepsilon} = k/n$. Hence,

$$P_{\text{success}}(n) = \sum_{k=0}^{\lfloor n\beta \rfloor} \binom{n}{k} \left(\frac{1}{4} + \alpha\right)^k \left(\frac{3}{4} - \alpha\right)^{n-k}. \quad (10)$$

The security requirements are $p > \beta$. The condition $p > \beta$ ensures that the bit-error rate induced by Alice's attack exceeds the detection threshold, meaning the attack leaves a detectable trace and is therefore likely to be rejected by the system. Consequently, the attack success probability $P_{\text{success}}(n)$ represents the probability that the system accepts the transmission despite an actual attack being present, i.e., the false negative rate. By Chernoff's bound for the binomial distribution, for $\beta < 1/4 + \alpha$,

$$P_{\text{success}}(n) \leq \exp \left[-n \left(\beta \ln \frac{\beta}{1/4 + \alpha} + (1 - \beta) \ln \frac{1 - \beta}{3/4 - \alpha} \right) \right]. \quad (11)$$

For clarity, the upper bound on Alice's attack success probability as a function of the number of quantum bits n is shown in Fig. 4. And, we adopt the Monte-Carlo method to conduct numerical simulations. For each value of qubit number n , we perform 20000 random sampling experiments obeying binomial distribution $k \sim \text{Binomial}(n, 0.30)$. We judge that the attack succeeds when $k \leq n\beta$ with $\beta = 0.06$. It can be seen that the simulation results fit well with the theoretical binomial expression, which confirms the consistency of the numerical implementation with the probabilistic model. The MATLAB code implementing the Monte-Carlo simulation is given in Appendix A.

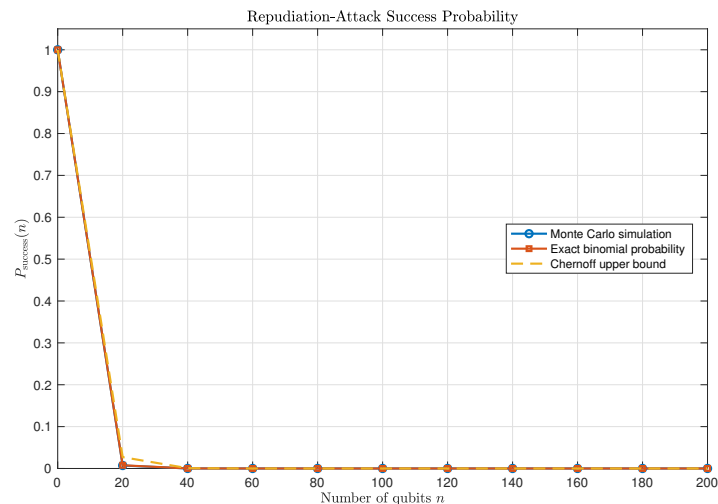


Figure 4. Attack success probability versus the number of qubits.

Finally, this constructed attack represents the strongest possible repudiation attempt, in which Alice deviates maximally in every step under her control. The analysis above covers three scenarios. If Bob is honest (Scenario 1), the protocol proceeds and Trent can trace the accepted message to Alice. If Bob's deviation prevents the protocol from proceeding (Scenario 2), Alice has no accepted signature to repudiate. If Bob's deviation inadvertently assists Alice (Scenario 3, probability $1/4$), the protocol proceeds and Trent's traceability still holds. Therefore, in all possible scenarios, Alice cannot successfully repudiate a signature that Bob has accepted.

5.2.2. Bob's attack simulation

We now analyze Bob's forgery attack under the assumption that Alice follows the protocol honestly. This corresponds to the standard unforgeability game in Section 4.3. If Alice were also dishonest, the analysis of Scenario 2 and Scenario 3 in the previous subsection applies symmetrically: Alice's deviation would either cause the protocol to abort or, with small probability, inadvertently assist Bob. In either case, Bob's forgery success remains bounded by the same arguments.

Bob's attack description. Bob has two attack opportunities. In the first stage (during protocol execution, Step (S3)-(S4)), Bob possesses $|p_2\rangle$ and $U(X_1)|p_1\rangle$. He may apply a non-identity operation G on $U(X_1)|p_1\rangle$ before teleporting it to Trent in Step (S4), or modify the plaintext copy $|p_2\rangle$. In the second stage (after protocol completion, Step (S7)), Bob holds $|p_2\rangle$ and $U(X_3)|p_1\rangle$, and may attempt to produce a forged signature.

Verification phase Analysis. For the first stage, if Bob applies $G \neq I$, the state reaching Trent becomes $U(X_2)GU(X_1)|p_1\rangle$. After Trent applies $U(X_1)$ and $U(X_2)$ in Step (S5), the recovered state is $U(X_1)GU(X_1)|p_1\rangle$ (up to a global phase). Trent compares this with $|p_3\rangle$ via the swap test. Since $|p_3\rangle$ was encrypted by Alice with K'_{AT} , Bob cannot alter it. The swap test detects the mismatch with probability at least $1 - 2^{-\lambda}$ whenever $G \neq I$. If Bob modifies $|p_2\rangle$ instead, the swap test in Step (S7) detects the mismatch with the same probability guarantee.

For the second stage, after recovering $|p_1\rangle$ and verifying it against $|p_2\rangle$, Bob already holds the valid message. To forge a different message, Bob would need to produce a new $|p^*\rangle$ and corresponding evidence that Trent accepts as originating from Alice. However, Trent retains the encrypted record $E_{K'_{AT}}|p_3\rangle$ and the particle pairs from both Alice and Bob, all protected by K_{AT} that Bob does not possess. Bob's knowledge of X_2 and X_3 provides no

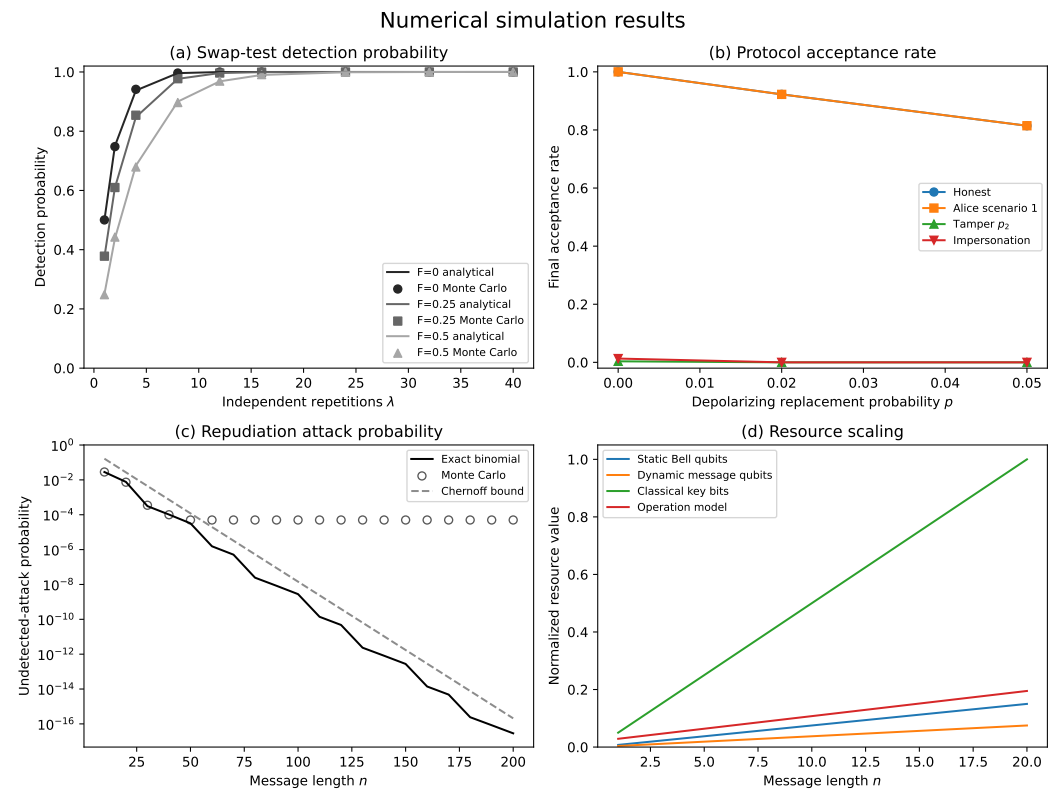


Figure 5. Numerical evaluation of the proposed scheme. (a) Analytical and Monte Carlo repeated swap-test detection; $1 - 2^{-\lambda}$ applies only to orthogonal inputs, whereas general overlap uses $1 - [(1 + F)/2]^\lambda$. (b) Protocol-level final acceptance under the stated state-level depolarizing replacement model. (c) Exact binomial, seeded Monte Carlo, and Chernoff results for the paper’s Alice-repudiation model. (d) Model-based, normalized resource accounting.

advantage, as these are random Bell measurement outcomes independent of the message content.

In summary, neither opportunity allows Bob to forge Alice’s signature with non-negligible probability, confirming the unforgeability of the proposed scheme.

5.3. Numerical simulations

We evaluated four protocol- and circuit-level quantities with fixed random seeds: repeated swap-test detection, honest and adversarial protocol acceptance, Alice’s repudiation probability model, and resource accounting. The simulations implement the three Bell resources of I2 and the repeated S5/S7 comparison rule. They illustrate the specified model and complement, but do not replace, the formal security analysis. The protocol-level execution calls I1–I2 and S1–S7 in order. Each teleportation leg retains its Bell state, Bell-measurement record, correction, and fidelity. The S5 and S7 comparisons use independent repeated-test outcomes. The results are presented in Fig. 5 and the complete Python code is provided in Appendix B.

These numerical results illustrate several properties of the proposed scheme under the stated models. First, the swap test achieves high detection probability by increasing the repetition count λ , and the analytical model is consistent with Monte Carlo simulations. Second, the protocol distinguishes honest execution from attacks with high reliability under the depolarizing noise model. Third, the probability that Alice’s repudiation attack remains undetected exhibits exponential decay with the message length n , reaching small values for $n \geq 200$. Fourth, all resource requirements exhibit the predicted linear trend in n , with quantum gate operations constituting the dominant cost. Together with the formal security

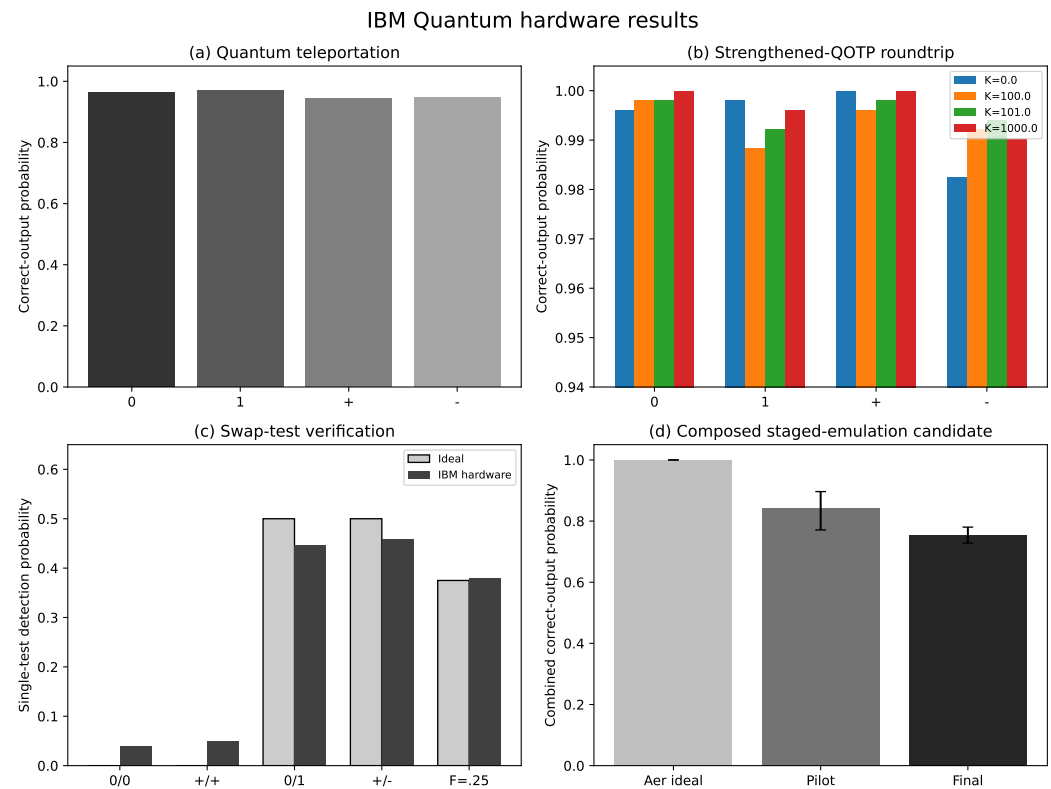


Figure 6. IBM hardware results for (a) teleportation correct-output probability, (b) strengthened-QOTP roundtrip probability, (c) swap-test detection for identical, orthogonal, and fixed-overlap states, and (d) faithful full-candidate correct-output probability. The candidate uses three explicit Bell resources; its Pilot Job d9o7s1fa5u8s73e2csjg used 128 shots and its Final Job d9o7tjva5u8s73e2cv9g used 1024 shots, yielding $773/1024=0.754883$. Error bars are Wilson 95% intervals where shown.

analysis, these results provide empirical support for the implementation feasibility of the proposed scheme.

Selected primitives and a composed candidate were executed on the 156-qubit IBM *ibm_fez* processor with Qiskit Runtime SamplerV2 in job mode. And, the complete Python code is provided in Appendix C. Fig. 6(a) shows the correct-output probability of standard quantum teleportation. Fig. 6(b) reports the roundtrip fidelity of the strengthened QOTP: a single-qubit message is encrypted and then decrypted, and the probability of recovering the original state is recorded. Fig. 6(c) presents the swap test detection capability for three types of input pairs: identical states, orthogonal states, and states with a fixed overlap. Fig. 6(d) gives the end-to-end correct-output probability of a faithful composed candidate that uses three explicit Bell resources and implements all circuit-representable operations of the proposed protocol. The candidate was prechecked with exact and Aer simulations before hardware submission. Its Pilot Job d9o7s1fa5u8s73e2csjg used 128 shots and its Final Job d9o7tjva5u8s73e2cv9g used 1024 shots, yielding $773/1024$ correct outputs.

These hardware results indicate that each individual primitive (teleportation, strengthened QOTP, swap test) can be executed on current superconducting hardware, and that a composed circuit implementing the circuit-representable operations of the protocol can be executed on a single chip with a correct-output probability of approximately 0.755. We note that this level of circuit fidelity is attributable to the accumulated gate and measurement errors in current superconducting quantum hardware. The protocol involves multiple rounds of Bell measurements, QOTP encryption and decryption, and swap tests, each contributing to the accumulated gate and measurement errors. As hardware gate fidelities

and coherence times continue to improve, the end-to-end correct-output probability is expected to increase accordingly.

In summary, the numerical simulations and hardware emulation presented in this subsection provide evidence for the practical potential of the proposed scheme. The swap test exhibits high detection probability under the depolarizing noise model, the protocol distinguishes normal execution from attacks with high reliability in the simulated scenarios, Alice's repudiation attack is bounded by the theoretically predicted limit, and all resource requirements scale linearly with message size. Together with the formal security analysis of Section 4.3, these results provide empirical support for the implementation feasibility of the proposed scheme.

6. Conclusion

This paper addressed the structural security challenges that arise when quantum search primitives are directly embedded into signature workflows. We identified structural vulnerabilities in a Grover-based quantum signature scheme and demonstrated that these vulnerabilities enable complete key recovery, Alice's disavowal, and Bob's existential forgery via man-in-the-middle attacks. To address these weaknesses, we proposed an arbitrated quantum signature scheme based on quantum teleportation and a strengthened quantum one-time pad. The security of the proposed scheme was analyzed through a formal adversarial model with well-defined security games for unforgeability and non-repudiation, and its resilience against replay, intercept-and-resend, man-in-the-middle, entanglement, and collective or coherent attacks was examined. Numerical simulations under depolarizing noise illustrated the behavior of the swap test as a verification primitive and the linear scaling of resource consumption.

Beyond signature schemes, future work may further extend quantum search algorithms to other quantum cryptographic protocols or multi-party quantum communication scenarios. Potential directions include routing optimization, repeater selection, and network diagnostics, where quantum speedups hold promise for reducing latency and enhancing resource efficiency. These avenues warrant further investigation within the context of future quantum networks.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (Nos. 62272208, 62572221).

Appendix A MATLAB code of Fig. 4

The following MATLAB code generates Fig. 4.

```
% Clear workspace
>> %% Parameter setting
alpha = 0.05;
beta  = 0.06;
p      = 0.25 + alpha;
trials = 20000;
n_list = 0:20:200;
sim_result = zeros(size(n_list));
theory_result = zeros(size(n_list));
for idx = 1:length(n_list)
    n = n_list(idx);
    threshold = floor(n * beta);
    success_case = 0;
    for t = 1:trials
        k = binornd(n,p);
        if(k <= threshold)
```

```

        success_case = success_case + 1;
    end
    end
    sim_result(idx) = success_case / trials;
    term1 = beta * log(beta / p);
    term2 = (1 - beta) * log((1 - beta) / (1 - p));
    theory_result(idx) = exp(-n * (term1 + term2));
end
%% Plot figure
figure('Position',[100,100,640,400])
plot(n_list,sim_result,'o-','LineWidth',2,'DisplayName','Monte-Carlo
    Numerical Simulation');
hold on;
plot(n_list,theory_result,'--','LineWidth',2,'DisplayName','Chernoff Bound
    Theoretical Upper-bound');
xlabel('Number of qubits $n$','Interpreter','latex');
ylabel('$P_{\{success\}}(n)$','Interpreter','latex');
title('Attack Success Probability: Simulation vs Theoretical Results');
legend;
grid on;
>>

```

Appendix B Python Code for Fig. 5

The following Python code generates all four subfigures presented in Fig. 5.

```

"""Continuous protocol-level implementation of I1-I2 and S1-S7."""
from __future__ import annotations
import argparse
import csv
import json
from dataclasses import dataclass, field
from math import comb, exp, floor, log, sqrt
from pathlib import Path
from typing import Any, Dict, Optional
import numpy as np
from qiskit import ClassicalRegister, QuantumCircuit, QuantumRegister
from qiskit.quantum_info import Statevector
# 1. Shared state and matrix definitions
I = np.eye(2, dtype=complex)
X = np.array([[0, 1], [1, 0]], dtype=complex)
Y = np.array([[0, -1j], [1j, 0]], dtype=complex)
Z = np.array([[1, 0], [0, -1]], dtype=complex)
H = np.array([[1, 1], [1, -1]], dtype=complex) / np.sqrt(2)
T_STRENGTHENED = np.sqrt(1j / 3) * (X - Y + Z)
STATES = {'0': np.array([1, 0], dtype=complex), '1': np.array([0, 1],
    dtype=complex), '+': np.array([1, 1], dtype=complex) / np.sqrt(2), '-':
    np.array([1, -1], dtype=complex) / np.sqrt(2)}
def normalized(vector) -> np.ndarray:
    array = np.asarray(vector, dtype=complex)
    return array / np.linalg.norm(array)
def fidelity(left, right) -> float:
    return float(abs(np.vdot(normalized(left), normalized(right))) ** 2)
def tensor(*items) -> np.ndarray:
    out = np.array([1.0 + 0j])
    for item in items:
        out = np.kron(out, np.asarray(item, dtype=complex))
    return out
def pauli_from_record(record: str) -> np.ndarray:
    return {'00': I, '01': X, '10': Z, '11': Z @ X}[record].copy()
@dataclass
class ComparisonRecord:
    stage: str
    fidelity: float

```

```

        p_accept_single: float
        p_accept_repeated: float
        outcome_accept: bool
        lambda_repetitions: int
        seed: int
        mode: str
    @dataclass
    class ProtocolState:
        message_label: str = '+'
        key_at: str = '0101101000111100'
        key_bt: str = '101001011100001110010110'
        key_parts: dict[str, str] = field(default_factory=dict)
        p1: np.ndarray | None = None
        p2: np.ndarray | None = None
        p3: np.ndarray | None = None
        bell_resources: Any = None
        x_records: dict[str, str] = field(default_factory=dict)
        teleportation: dict[str, Any] = field(default_factory=dict)
        encrypted_records: dict[str, Any] = field(default_factory=dict)
        recovered_records: dict[str, str] = field(default_factory=dict)
        p3_encrypted: np.ndarray | None = None
        p3_recovered: np.ndarray | None = None
        trent_state: np.ndarray | None = None
        bob_state: np.ndarray | None = None
        comparisons: list[ComparisonRecord] = field(default_factory=list)
        trent_accept: bool = False
        bob_accept: bool = False
        final_accept: bool = False
        audit_trail: list[dict[str, Any]] = field(default_factory=list)
        def log(self, stage: str, **details) -> None:
            self.audit_trail.append({'stage': stage, **details})
    # 2. Key partition
    def partition_keys(key_at: str, key_bt: str) -> dict[str, str]:
        if len(key_at) != 16 or set(key_at) - {'0', '1'}:
            raise ValueError('K_AT must be 16 bits')
        if len(key_bt) != 24 or set(key_bt) - {'0', '1'}:
            raise ValueError('K_BT must be 24 bits')
        return {'K_AT_prime': key_at[:4], 'K_AT_double': key_at[4:8],
                'K_AT_triple': key_at[8:16], 'K_BT_prime': key_bt[:4],
                'K_BT_double': key_bt[4:12], 'K_BT_triple': key_bt[12:16],
                'K_BT_fourth': key_bt[16:24]}
    # 3. Three Bell resources
    def bell_phi_plus() -> np.ndarray:
        return np.array([1, 0, 0, 1], dtype=complex) / np.sqrt(2)
    @dataclass
    class BellPairResource:
        pair_id: str
        sender: str
        receiver: str
        stage: str
        state: np.ndarray
        consumed: bool = False
        measurement_bits: str | None = None
        correction: str | None = None
    @dataclass
    class BellResources:
        alice_bob_pair: BellPairResource
        bob_trent_pair: BellPairResource
        trent_bob_pair: BellPairResource
    def prepare_bell_pair(pair_id: str, sender: str, receiver: str, stage: str)
    -> BellPairResource:
        return BellPairResource(pair_id, sender, receiver, stage,
                                bell_phi_plus())

```

```

def prepare_three_bell_pairs() -> BellResources:
    return BellResources(prepare_bell_pair('AB_S2', 'Alice', 'Bob', 'S2'),
        prepare_bell_pair('BT_S4', 'Bob', 'Trent', 'S4'),
        prepare_bell_pair('TB_S6', 'Trent', 'Bob', 'S6'))
# 4. Strengthened-QOTP
def encryption_unitary(key4: str) -> np.ndarray:
    if len(key4) != 4 or set(key4) - {'0', '1'}:
        raise ValueError('key block must be 4 bits')
    a, b, c, d = map(int, key4)
    return np.linalg.matrix_power(X, a) @ np.linalg.matrix_power(Z, b) @
        T_STRENGTHENED @ np.linalg.matrix_power(X, c) @
        np.linalg.matrix_power(Z, d)
def encrypt_state(state, key4: str) -> np.ndarray:
    return encryption_unitary(key4) @ np.asarray(state, dtype=complex)
def decrypt_state(state, key4: str) -> np.ndarray:
    return encryption_unitary(key4).conj().T @ np.asarray(state,
        dtype=complex)
def encrypt_two_qubit_state(state, key8: str) -> np.ndarray:
    if len(key8) != 8:
        raise ValueError('two-qubit record key must be 8 bits')
    return tensor(encryption_unitary(key8[:4]),
        encryption_unitary(key8[4:])) @ np.asarray(state, dtype=complex)
def decrypt_two_qubit_state(state, key8: str) -> np.ndarray:
    unitary = tensor(encryption_unitary(key8[:4]),
        encryption_unitary(key8[4:]))
    return unitary.conj().T @ np.asarray(state, dtype=complex)
# 5. Teleportation
@dataclass
class TeleportationResult:
    stage: str
    pair_id: str
    measurement_bits: str
    branch_probability: float
    receiver_before_correction: np.ndarray
    receiver_after_correction: np.ndarray
    correction: str
    fidelity: float
def teleportation_branch(message, bell_resource, record: str, stage: str) ->
    TeleportationResult:
    if record not in {'00', '01', '10', '11'}:
        raise ValueError('record must be two bits')
    qc = QuantumCircuit(3)
    qc.initialize(normalized(message), 0)
    qc.h(1)
    qc.cx(1, 2)
    qc.cx(0, 1)
    qc.h(0)
    sv = Statevector.from_instruction(qc).data
    m0, m1 = map(int, record)
    receiver = np.array([sv[m0 + 2 * m1], sv[m0 + 2 * m1 + 4]],
        dtype=complex)
    probability = float(np.vdot(receiver, receiver).real)
    receiver = normalized(receiver)
    correction = pauli_from_record(record)
    corrected = correction @ receiver
    bell_resource.consumed = True
    bell_resource.measurement_bits = record
    bell_resource.correction = record
    return TeleportationResult(stage, bell_resource.pair_id, record,
        probability, receiver, corrected, record, fidelity(message,
        corrected))
def alice_to_bob_teleportation(message, resources, record='01'):

```

```

        return teleportation_branch(message, resources.alice_bob_pair, record,
                                     'S2')
def bob_to_trent_teleportation(message, resources, record='10'):
    return teleportation_branch(message, resources.bob_trent_pair, record,
                                 'S4')
def trent_to_bob_teleportation(message, resources, record='11'):
    return teleportation_branch(message, resources.trent_bob_pair, record,
                                 'S6')
# 6. Repeated swap-test verification
def swap_test_accept_probability(F: float) -> float:
    return (1 + float(F)) / 2
def repeated_swap_test_accept_probability(F: float, lam: int) -> float:
    return swap_test_accept_probability(F) ** int(lam)
def repeated_swap_test_detection_probability(F: float, lam: int) -> float:
    return 1 - repeated_swap_test_accept_probability(F, lam)
def sample_repeated_swap_test(F: float, lam: int, rng) -> bool:
    return bool(rng.random() < repeated_swap_test_accept_probability(F, lam))
def compare_states(left, right, lam, rng, mode='repeated_swap_test'):
    F = fidelity(left, right)
    single = swap_test_accept_probability(F)
    repeated = single ** lam
    accepted = F > 1 - 1e-10 if mode == 'exact_reference' else
        sample_repeated_swap_test(F, lam, rng)
    return (F, single, repeated, accepted)
def build_swap_test_circuit(left_label='0', right_label='0',
                             name='swap_test'):
    q = QuantumRegister(3, 'q')
    c = ClassicalRegister(1, 'accept')
    qc = QuantumCircuit(q, c, name=name)
    for idx, label in ((1, left_label), (2, right_label)):
        if label == '1':
            qc.x(idx)
        elif label == '+':
            qc.h(idx)
        elif label == '-':
            qc.x(idx)
            qc.h(idx)
    qc.h(0)
    qc.cswap(0, 1, 2)
    qc.h(0)
    qc.measure(0, 0)
    return qc
# 7. Encrypted records
def record_basis_state(record: str) -> np.ndarray:
    out = np.zeros(4, dtype=complex)
    out[int(record, 2)] = 1
    return out
def encrypt_computational_record(record: str, key8: str) -> np.ndarray:
    return encrypt_two_qubit_state(record_basis_state(record), key8)
def recover_computational_record(cipher, key8: str) -> str:
    state = decrypt_two_qubit_state(cipher, key8)
    return f'{int(np.argmax(abs(state) ** 2)):02b}'
def encrypted_bell_record(record: str, key4: str) -> np.ndarray:
    operator = encryption_unitary(key4) @ pauli_from_record(record)
    return tensor(I, operator) @ bell_phi_plus()
def recover_bell_record(cipher, key4: str) -> str:
    for record in ('00', '01', '10', '11'):
        if fidelity(cipher, encrypted_bell_record(record, key4)) > 1 - 1e-10:
            return record
    return 'UNRESOLVED'
# 8. Attack injection used by protocol execution
def _flip_first(cipher):
    array = np.asarray(cipher, dtype=complex)

```

```

        return tensor(X, np.eye(2)) @ array if array.size == 4 else
            array[:: -1].copy()
def apply_attack(state, name: str, stage: str):
    if name == 'none':
        return state
    if stage == 'after_s3':
        if name == 'tamper_X1_bell_record':
            state.encrypted_records['X1_bell'] =
                _flip_first(state.encrypted_records['X1_bell'])
        elif name == 'tamper_X1_encrypted_record':
            state.encrypted_records['X1_comp'] =
                _flip_first(state.encrypted_records['X1_comp'])
        elif name == 'tamper_p2':
            state.p2 = Z @ state.p2
        elif name == 'tamper_p3':
            state.p3_encrypted = state.p3_encrypted[:: -1].copy()
        elif name == 'impersonation_attempt':
            state.p3_encrypted = np.array([1, 0], dtype=complex)
    if stage == 'after_s4':
        if name == 'tamper_X2_bell_record':
            state.encrypted_records['X2_bell'] =
                _flip_first(state.encrypted_records['X2_bell'])
        elif name == 'tamper_X2_encrypted_record':
            state.encrypted_records['X2_comp'] =
                _flip_first(state.encrypted_records['X2_comp'])
        elif name == 'replay_mismatched_record':
            state.encrypted_records['X2_comp'] =
                state.encrypted_records['X1_comp'].copy()
    if stage == 'after_s6':
        if name == 'tamper_X3_bell_record':
            state.encrypted_records['X3_bell'] =
                _flip_first(state.encrypted_records['X3_bell'])
        elif name == 'tamper_X3_encrypted_record':
            state.encrypted_records['X3_comp'] =
                _flip_first(state.encrypted_records['X3_comp'])
    state.log('ATTACK', name=name, injection_stage=stage)
    return state
# 9. Protocol steps I1-I2
def execute_i1(state):
    state.key_parts = partition_keys(state.key_at, state.key_bt)
    state.log('I1', key_at_bits=16, key_bt_bits=24, partitions={k: len(v)
        for k, v in state.key_parts.items()})
    return state
def execute_i2(state):
    state.bell_resources = prepare_three_bell_pairs()
    state.log('I2', pairs=['AB_S2', 'BT_S4', 'TB_S6'], static_bell_qubits=6)
    return state
# 10. Protocol steps S1-S3
def execute_s1(state):
    template = STATES[state.message_label]
    state.p1 = template.copy()
    state.p2 = template.copy()
    state.p3 = template.copy()
    state.log('S1', copies=3, preparation='independent from known classical
        description')
    return state
def execute_s2(state, record='01'):
    result = alice_to_bob_teleportation(state.p1, state.bell_resources,
        record)
    state.x_records['X1'] = record
    state.teleportation['S2'] = result
    state.bob_state = result.receiver_before_correction.copy()

```

```

        state.log('S2', pair=result.pair_id, X1=record,
                  branch_probability=result.branch_probability,
                  corrected_fidelity=result.fidelity)
        return state
def execute_s3(state):
    p = state.key_parts
    x1 = state.x_records['X1']
    state.p3_encrypted = encrypt_state(state.p3, p['K_AT_prime'])
    state.encrypted_records['X1_bell'] = encrypted_bell_record(x1,
                                                                p['K_AT_double'])
    state.encrypted_records['X1_comp'] = encrypt_computational_record(x1,
                                                                      p['K_AT_triple'])
    state.log('S3', objects=['p2', 'E_KAT_prime(p3)', 'encrypted_X1_bell',
                             'encrypted_X1_comp'])
    return state
# 11. Protocol step S4
def execute_s4(state, record='10'):
    result = bob_to_trent_teleportation(state.bob_state,
                                         state.bell_resources, record)
    state.x_records['X2'] = record
    state.teleportation['S4'] = result
    state.trent_state = result.receiver_before_correction.copy()
    p = state.key_parts
    state.encrypted_records['X2_bell'] = encrypted_bell_record(record,
                                                                p['K_BT_prime'])
    state.encrypted_records['X2_comp'] =
        encrypt_computational_record(record, p['K_BT_double'])
    state.log('S4', pair=result.pair_id, X2=record,
              branch_probability=result.branch_probability)
    return state
# 12. Protocol steps S5-S6
def execute_s5(state, lam, rng, mode='repeated_swap_test', seed=0):
    p = state.key_parts
    recovered = {'X1_bell':
                 recover_bell_record(state.encrypted_records['X1_bell'],
                                     p['K_AT_double']), 'X1_comp':
                 recover_computational_record(state.encrypted_records['X1_comp'],
                                               p['K_AT_triple']), 'X2_bell':
                 recover_bell_record(state.encrypted_records['X2_bell'],
                                     p['K_BT_prime']), 'X2_comp':
                 recover_computational_record(state.encrypted_records['X2_comp'],
                                               p['K_BT_double'])}
    state.recovered_records.update(recovered)
    records_ok = recovered['X1_bell'] == recovered['X1_comp'] and
                 recovered['X2_bell'] == recovered['X2_comp']
    x1 = recovered['X1_comp'] if recovered['X1_comp'] in {'00', '01', '10',
                                                         '11'} else '00'
    x2 = recovered['X2_comp'] if recovered['X2_comp'] in {'00', '01', '10',
                                                         '11'} else '00'
    recovered_p1 = pauli_from_record(x1).conj().T @
                  pauli_from_record(x2).conj().T @ state.trent_state
    state.p3_recovered = decrypt_state(state.p3_encrypted, p['K_AT_prime'])
    F, single, repeated, accepted = compare_states(recovered_p1,
                                                    state.p3_recovered, lam, rng, mode)
    state.comparisons.append(ComparisonRecord('S5', F, single, repeated,
                                              accepted, lam, seed, mode))
    state.trent_accept = bool(records_ok and accepted)
    state.trent_state = recovered_p1
    state.log('S5', records_ok=records_ok, fidelity=F,
              p_accept_single=single, p_accept_repeated=repeated,
              outcome=accepted, trent_accept=state.trent_accept)
    return state
def execute_s6(state, record='11'):

```

```

        result = trent_to_bob_teleportation(state.trent_state,
            state.bell_resources, record)
        state.x_records['X3'] = record
        state.teleportation['S6'] = result
        state.bob_state = result.receiver_before_correction.copy()
        p = state.key_parts
        state.encrypted_records['X3_bell'] = encrypted_bell_record(record,
            p['K_BT_triple'])
        state.encrypted_records['X3_comp'] =
            encrypt_computational_record(record, p['K_BT_fourth'])
        state.log('S6', pair=result.pair_id, X3=record,
            branch_probability=result.branch_probability)
        return state
# 13. Protocol step S7
def execute_s7(state, lam, rng, mode='repeated_swap_test', seed=0):
    p = state.key_parts
    xb = recover_bell_record(state.encrypted_records['X3_bell'],
        p['K_BT_triple'])
    xc = recover_computational_record(state.encrypted_records['X3_comp'],
        p['K_BT_fourth'])
    state.recovered_records.update({'X3_bell': xb, 'X3_comp': xc})
    records_ok = xb == xc
    record = xc if xc in {'00', '01', '10', '11'} else '00'
    state.bob_state = pauli_from_record(record).conj().T @ state.bob_state
    F, single, repeated, accepted = compare_states(state.bob_state,
        state.p2, lam, rng, mode)
    state.comparisons.append(ComparisonRecord('S7', F, single, repeated,
        accepted, lam, seed, mode))
    state.bob_accept = bool(records_ok and accepted)
    state.final_accept = bool(state.trent_accept and state.bob_accept)
    state.log('S7', records_ok=records_ok, fidelity=F,
        p_accept_single=single, p_accept_repeated=repeated,
        outcome=accepted, bob_accept=state.bob_accept,
        final_accept=state.final_accept)
    return state
# 14. Complete protocol execution
def run_protocol_v7(message_label='+', attack='none',
    comparison_mode='repeated_swap_test', lambda_repetitions=8,
    seed=20260803):
    state = ProtocolState(message_label=message_label)
    rng = np.random.default_rng(seed)
    execute_i1(state)
    execute_i2(state)
    execute_s1(state)
    execute_s2(state)
    if attack == 'alice_scenario_1':
        true = state.x_records['X1']
        sent = next((x for x in ('00', '01', '10', '11') if x != true))
        state.x_records['X1'] = sent
        state.p2 = pauli_from_record(sent) @ pauli_from_record(true) @
            state.p1
        state.p3 = state.p2.copy()
        state.log('ALICE_SCENARIO_1', X1_true=true, X1_sent=sent,
            traceability_relation='p2=p3=U(X1_sent)U(X1_true)p1')
    execute_s3(state)
    apply_attack(state, attack, 'after_s3')
    execute_s4(state)
    apply_attack(state, attack, 'after_s4')
    execute_s5(state, lambda_repetitions, rng, comparison_mode, seed)
    execute_s6(state)
    apply_attack(state, attack, 'after_s6')
    execute_s7(state, lambda_repetitions, rng, comparison_mode, seed + 1)
    return state

```

```

def run_honest_protocol(**kwargs):
    return run_protocol_v7(attack='none', **kwargs)
def run_protocol_with_attack(attack, **kwargs):
    return run_protocol_v7(attack=attack, **kwargs)
def _write_protocol_summary(path: Path, attack: str) -> None:
    state = run_protocol_with_attack(attack,
        comparison_mode="exact_reference", seed=20260803)
    row = {
        "attack": attack,
        "message": state.message_label,
        "X1": state.x_records.get("X1"),
        "X2": state.x_records.get("X2"),
        "X3": state.x_records.get("X3"),
        "trent_accept": state.trent_accept,
        "bob_accept": state.bob_accept,
        "final_accept": state.final_accept,
    }
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(row))
        writer.writeheader(); writer.writerow(row)
# 2. Grover-based scheme
def basis_state(index: int, dimension: int=4) -> np.ndarray:
    state = np.zeros(dimension, dtype=complex)
    state[index % dimension] = 1.0
    return state
def public_message_state(dimension: int=4) -> np.ndarray:
    return np.ones(dimension, dtype=complex) / np.sqrt(dimension)
def message_state_from_bits(m: str) -> np.ndarray:
    if m not in {'00', '01', '10', '11'}:
        raise ValueError("Grover message bits must be one of '00', '01',
            '10', '11'.")
    plus = np.array([1.0, 1.0], dtype=complex) / np.sqrt(2)
    minus = np.array([1.0, -1.0], dtype=complex) / np.sqrt(2)
    first = minus if m[0] == '0' else plus
    second = minus if m[1] == '0' else plus
    state = np.kron(first, second)
    return state / np.linalg.norm(state)
def signing_unitary(key_index: int, dimension: int=4) -> np.ndarray:
    key = basis_state(key_index, dimension)
    return np.eye(dimension, dtype=complex) - 2.0 * np.outer(key, key.conj())
def verification_unitary(message_state: np.ndarray) -> np.ndarray:
    message_state = message_state / np.linalg.norm(message_state)
    return 2.0 * np.outer(message_state, message_state.conj()) -
        np.eye(len(message_state), dtype=complex)
def equivalent_key_index(state: np.ndarray) -> int:
    return int(np.argmax(np.abs(state) ** 2))
@dataclass
class GroverStepwiseState:
    message_label: str = '00'
    dimension: int = 4
    rng: np.random.Generator = field(default_factory=lambda:
        np.random.default_rng(2026))
    keys: Dict[str, int] = field(default_factory=dict)
    objects: Dict[str, object] = field(default_factory=dict)
    transcript: list[str] = field(default_factory=list)
    accepted_by_tc: Optional[bool] = None
    accepted_by_bob: Optional[bool] = None
    recovered_keys: Dict[str, int] = field(default_factory=dict)
def step_AT1_alice_tc_key_setup(state: GroverStepwiseState) -> None:
    state.objects['M_AT'] = message_state_from_bits(state.message_label)
    state.keys['K_AT'] = int(state.rng.integers(0, state.dimension))
    state.transcript.append(f'AT1: public message m={state.message_label}
        and Alice-TC key initialized.')

```

```

def step_AT2_construct_alice_signing_unitary(state: GroverStepwiseState) ->
    None:
        state.objects['U_SAT'] = signing_unitary(state.keys['K_AT'],
            state.dimension)
        state.transcript.append('AT2: Alice signing unitary U_SAT constructed.')
def step_AT3_generate_alice_signature(state: GroverStepwiseState) -> None:
    state.objects['alice_signature'] = state.objects['U_SAT'] @
        state.objects['M_AT']
    state.transcript.append('AT3: Alice signature state U_SAT|M> generated.')
def step_AT4_AT7_transmit_to_tc_with_decoy_proxy(state: GroverStepwiseState,
    attack: str='none') -> None:
    state.objects['tc_received_signature'] =
        state.objects['alice_signature'].copy()
    state.objects['channel_check_AT'] = attack != 'channel_block'
    state.objects['decoy_proxy_AT'] = 'AT4-AT7 channel/transcript proxy'
    state.transcript.append('AT4-AT7: Alice-to-TC decoy-assisted channel
        represented by proxy.')
def step_AT8_tc_verify_and_recover_key(state: GroverStepwiseState) -> None:
    verifier = verification_unitary(state.objects['M_AT'])
    recovered_state = verifier @ state.objects['tc_received_signature']
    recovered = equivalent_key_index(recovered_state)
    state.objects['U_V_AT'] = verifier
    state.objects['recovered_state_AT'] = recovered_state
    state.recovered_keys['K_AT'] = recovered
    state.accepted_by_tc = bool(state.objects.get('channel_check_AT', False)
        and recovered == state.keys['K_AT'])
    state.transcript.append('AT8: TC recovered and checked K_AT.')
def step_TB1_tc_bob_key_setup(state: GroverStepwiseState) -> None:
    state.objects['M_TB'] = message_state_from_bits(state.message_label)
    state.keys['K_TB'] = int(state.rng.integers(0, state.dimension))
    state.transcript.append(f'TB1: public message m={state.message_label}
        and TC-Bob key initialized.')
def step_TB2_construct_tc_signing_unitary(state: GroverStepwiseState) ->
    None:
        state.objects['U_STB'] = signing_unitary(state.keys['K_TB'],
            state.dimension)
        state.transcript.append('TB2: TC signing unitary U_STB constructed.')
def step_TB3_generate_tc_signature(state: GroverStepwiseState) -> None:
    state.objects['tc_signature'] = state.objects['U_STB'] @
        state.objects['M_TB']
    state.transcript.append('TB3: TC signature state U_STB|M> generated.')
def step_TB4_TB7_transmit_to_bob_with_decoy_proxy(state:
    GroverStepwiseState, attack: str='none') -> None:
    state.objects['bob_received_signature'] =
        state.objects['tc_signature'].copy()
    state.objects['channel_check_TB'] = attack != 'channel_block'
    state.objects['decoy_proxy_TB'] = 'TB4-TB7 channel/transcript proxy'
    state.transcript.append('TB4-TB7: TC-to-Bob decoy-assisted channel
        represented by proxy.')
def step_TB8_bob_verify_and_recover_key(state: GroverStepwiseState) -> None:
    verifier = verification_unitary(state.objects['M_TB'])
    recovered_state = verifier @ state.objects['bob_received_signature']
    recovered = equivalent_key_index(recovered_state)
    state.objects['U_V_TB'] = verifier
    state.objects['recovered_state_TB'] = recovered_state
    state.recovered_keys['K_TB'] = recovered
    state.accepted_by_bob = bool(state.objects.get('channel_check_TB',
        False) and recovered == state.keys['K_TB'])
    state.transcript.append('TB8: Bob recovered and checked K_TB.')
def run_at_protocol(seed: int=2026, message_bits: str='00') ->
    GroverStepwiseState:
    state = GroverStepwiseState(message_label=message_bits,
        rng=np.random.default_rng(seed))

```

```

        step_AT1_alice_tc_key_setup(state)
        step_AT2_construct_alice_signing_unitary(state)
        step_AT3_generate_alice_signature(state)
        step_AT4_AT7_transmit_to_tc_with_decoy_proxy(state)
        step_AT8_tc_verify_and_recover_key(state)
        return state
def run_tb_protocol(seed: int=2026, message_bits: str='00') ->
GroverStepwiseState:
    state = GroverStepwiseState(message_label=message_bits,
        rng=np.random.default_rng(seed))
    step_TB1_tc_bob_key_setup(state)
    step_TB2_construct_tc_signing_unitary(state)
    step_TB3_generate_tc_signature(state)
    step_TB4_TB7_transmit_to_bob_with_decoy_proxy(state)
    step_TB8_bob_verify_and_recover_key(state)
    return state
def recover_public_key_from_signature(signature: np.ndarray, message_state:
np.ndarray) -> int:
    return equivalent_key_index(verification_unitary(message_state) @
        signature)
# 3. Grover key recovery and recovered-key forgery
def run_key_recovery_and_forgery(message_bits: str, key_index: int):
    message = message_state_from_bits(message_bits)
    signature = signing_unitary(key_index) @ message
    recovered_state = verification_unitary(message) @ signature
    recovered = int(np.argmax(abs(recovered_state) ** 2))
    recovery_fidelity = float(abs(np.vdot(basis_state(recovered),
        recovered_state)) ** 2)
    forged = signing_unitary(recovered) @ message
    check = verification_unitary(message) @ forged
    return {'message': message_bits, 'true_key': key_index, 'recovered_key':
        recovered, 'key_recovery_rate': float(recovered == key_index),
        'recovery_fidelity': recovery_fidelity, 'forged_accept_rate':
        float(int(np.argmax(abs(check) ** 2)) == recovered)}
# 4. Attack scenario catalogue
TARGETED_ATTACKS = ('tamper_X1_bell_record', 'tamper_X1_encrypted_record',
    'tamper_X2_bell_record', 'tamper_X2_encrypted_record',
    'tamper_X3_bell_record', 'tamper_X3_encrypted_record', 'tamper_p2',
    'tamper_p3', 'replay_mismatched_record', 'impersonation_attempt')
# 5. Alice repudiation probability model
def exact_binomial_probability(n: int, alpha: float, beta: float) -> float:
    p = 0.25 + alpha
    kmax = floor(n * beta)
    return sum((comb(n, k) * p ** k * (1 - p) ** (n - k) for k in range(kmax
        + 1)))
def chernoff_upper_bound(n: int, alpha: float, beta: float) -> float:
    p = 0.25 + alpha
    q = beta
    if q <= 0:
        return (1 - p) ** n
    divergence = q * log(q / p) + (1 - q) * log((1 - q) / (1 - p))
    return exp(-n * divergence)
def monte_carlo_probability(n: int, alpha: float, beta: float, trials: int,
rng) -> tuple[float, int]:
    successes = int(np.count_nonzero(rng.binomial(n, 0.25 + alpha,
        size=trials) <= floor(n * beta)))
    return (successes / trials, successes)
# 6. Result generation
def generate_attack_results(output_dir: Path, seed: int = 20260803) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    grover_rows = [run_key_recovery_and_forgery(message, key) for message in
        ("00", "01", "10", "11") for key in range(4)]

```

```

with (output_dir / "grover_attack_results.csv").open("w", newline="",
encoding="utf-8") as handle:
    writer = csv.DictWriter(handle, fieldnames=list(grover_rows[0]));
    writer.writeheader(); writer.writerows(grover_rows)
attacks = [
    "none", "tamper_X1_bell_record", "tamper_X1_encrypted_record",
    "tamper_X2_bell_record", "tamper_X2_encrypted_record",
    "tamper_X3_bell_record", "tamper_X3_encrypted_record",
    "tamper_p2", "tamper_p3", "replay_mismatched_record",
    "impersonation_attempt",
]
protocol_rows = []
for attack in attacks:
    state = run_protocol_with_attack(attack,
comparison_mode="exact_reference", seed=seed)
    protocol_rows.append({"attack": attack, "trent_accept":
state.trent_accept, "bob_accept": state.bob_accept,
"final_accept": state.final_accept})
with (output_dir / "targeted_attack_results.csv").open("w", newline="",
encoding="utf-8") as handle:
    writer = csv.DictWriter(handle, fieldnames=list(protocol_rows[0]));
    writer.writeheader(); writer.writerows(protocol_rows)
rng = np.random.default_rng(seed)
repudiation_rows = []
for n in range(10, 201, 10):
    monte_carlo, count = monte_carlo_probability(n, 0.05, 0.06, 20000,
rng)
    repudiation_rows.append({"n": n, "alpha": 0.05, "beta": 0.06,
"trials": 20000, "success_count": count,
"monte_carlo_probability": monte_carlo,
"exact_binomial_probability": exact_binomial_probability(n,
0.05, 0.06),
"chernoff_upper_bound": chernoff_upper_bound(n, 0.05, 0.06)})
with (output_dir / "alice_repudiation_results.csv").open("w",
newline="", encoding="utf-8") as handle:
    writer = csv.DictWriter(handle,
fieldnames=list(repudiation_rows[0])); writer.writeheader();
    writer.writerows(repudiation_rows)
if False and __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Run the complete attack
benchmark locally.")
    parser.add_argument("--output-dir", type=Path,
default=Path("attack_results"))
    parser.add_argument("--seed", type=int, default=20260803)
    args = parser.parse_args(); generate_attack_results(args.output_dir,
args.seed); print(args.output_dir)
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
# Numerical data generation for the four-panel figure
import hashlib
NUMERICAL_SEED = 20260803
def generate_swap_test_detection_data(path: Path, trials: int = 20000) ->
None:
    rng = np.random.default_rng(NUMERICAL_SEED); rows = []
    for F in (0.0, 0.25, 0.5):
        for lam in (1, 2, 4, 8, 12, 16, 24, 32, 40):
            analytical = repeated_swap_test_detection_probability(F, lam)
            detected = int(np.count_nonzero(rng.random(trials) < analytical))
            rows.append({"fidelity": F, "lambda": lam, "trials": trials,
"analytical_detection_probability": analytical,

```

```

        "monte_carlo_detection_probability": detected /
            trials})
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]));
        writer.writeheader(); writer.writerows(rows)
def generate_protocol_acceptance_data(path: Path, trials: int = 2000, lam:
    int = 8) -> None:
    rows = []
    for attack in ("none", "alice_scenario_1", "tamper_p2",
        "impersonation_attempt"):
        for noise in (0.0, 0.02, 0.05):
            final_accepts = 0
            for index in range(trials):
                state = run_protocol_v7(attack=attack,
                    comparison_mode="repeated_swap_test",
                    lambda_repetitions=lam, seed=NUMERICAL_SEED + index)
                if noise:
                    rng = np.random.default_rng(NUMERICAL_SEED + index + 91)
                    repeated_outcomes = [rng.random() < ((1 + ((1 - noise) *
                        item.fidelity + noise / 2)) / 2) ** lam for item in
                        state.comparisons]
                    state.final_accept = state.final_accept and
                        all(repeated_outcomes)
                    final_accepts += int(state.final_accept)
            rows.append({"attack": attack, "noise_parameter": noise,
                "lambda": lam, "trials": trials,
                    "final_accept_count": final_accepts,
                        "final_accept_rate": final_accepts / trials})
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]));
        writer.writeheader(); writer.writerows(rows)
def generate_repudiation_data(path: Path, trials: int = 20000) -> None:
    rng = np.random.default_rng(NUMERICAL_SEED); rows = []
    for n in range(10, 201, 10):
        monte_carlo, success_count = monte_carlo_probability(n, 0.05, 0.06,
            trials, rng)
        rows.append({"n": n, "alpha": 0.05, "beta": 0.06, "trials": trials,
            "success_count": success_count,
                "monte_carlo_probability": monte_carlo,
                    "exact_binomial_probability":
                        exact_binomial_probability(n, 0.05, 0.06),
                            "chernoff_upper_bound": chernoff_upper_bound(n, 0.05,
                                0.06)})
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]));
        writer.writeheader(); writer.writerows(rows)
def generate_resource_data(path: Path, lam: int = 8) -> None:
    rows = [{"n": n, "lambda": lam, "static_bell_qubits": 6 * n,
        "dynamic_message_qubits": 3 * n,
            "classical_key_bits_model": 40 * n,
                "normalized_operation_model": 7 * n + 2 * lam}
        for n in range(1, 21)]
    with path.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]));
        writer.writeheader(); writer.writerows(rows)
def generate_complete_numerical_figure(data_dir: Path, output_path: Path) ->
    None:
    swap = pd.read_csv(data_dir / "swap_test_detection.csv")
    acceptance = pd.read_csv(data_dir / "protocol_acceptance.csv")
    repudiation = pd.read_csv(data_dir / "alice_repudiation_probability.csv")
    resources = pd.read_csv(data_dir / "resource_scaling.csv")
    fig, axes = plt.subplots(2, 2, figsize=(12, 9))

```

```

for F, color, marker in ((0.0, "0.15", "o"), (0.25, "0.4", "s"), (0.5,
    "0.65", "^")):
    part = swap[swap["fidelity"] == F]
    axes[0, 0].plot(part["lambda"],
        part["analytical_detection_probability"], color=color,
        label=f"F={F:g} analytical")
    axes[0, 0].scatter(part["lambda"],
        part["monte_carlo_detection_probability"], color=color,
        marker=marker, label=f"F={F:g} Monte Carlo")
axes[0, 0].set(title="(a) Swap-test detection probability",
    xlabel=r"Independent repetitions  $\lambda$ ", ylabel="Detection
    probability", ylim=(0, 1.02)); axes[0, 0].legend(fontsize=8)
for attack, label, marker in (("none", "Honest", "o"),
    ("alice_scenario_1", "Alice scenario 1", "s"), ("tamper_p2",
    r"Tamper  $p_2$ ", "^"), ("impersonation_attempt", "Impersonation",
    "v")):
    part = acceptance[acceptance["attack"] == attack]
    axes[0, 1].plot(part["noise_parameter"], part["final_accept_rate"],
        marker=marker, label=label)
axes[0, 1].set(title="(b) Protocol acceptance rate",
    xlabel=r"Depolarizing replacement probability  $p$ ", ylabel="Final
    acceptance rate", ylim=(-0.02, 1.02)); axes[0, 1].legend(fontsize=8)
axes[1, 0].semilogy(repudiation["n"],
    repudiation["exact_binomial_probability"], color="black",
    label="Exact binomial")
axes[1, 0].scatter(repudiation["n"],
    repudiation["monte_carlo_probability"].clip(lower=1 /
    repudiation["trials"]), facecolors="none", edgecolors="0.4",
    label="Monte Carlo")
axes[1, 0].semilogy(repudiation["n"],
    repudiation["chernoff_upper_bound"], "--", color="0.6",
    label="Chernoff bound")
axes[1, 0].set(title="(c) Repudiation attack probability",
    xlabel=r"Message length  $n$ ", ylabel="Undetected-attack
    probability"); axes[1, 0].legend(fontsize=8)
maximum = max(resources[column].max() for column in
    ("static_bell_qubits", "dynamic_message_qubits",
    "classical_key_bits_model", "normalized_operation_model"))
for column, label in (("static_bell_qubits", "Static Bell qubits"),
    ("dynamic_message_qubits", "Dynamic message qubits"),
    ("classical_key_bits_model", "Classical key bits"),
    ("normalized_operation_model", "Operation model")):
    axes[1, 1].plot(resources["n"], resources[column] / maximum,
        label=label)
axes[1, 1].set(title="(d) Resource scaling", xlabel=r"Message length
     $n$ ", ylabel="Normalized resource value", ylim=(0, 1.05)); axes[1,
    1].legend(fontsize=8)
fig.suptitle("Numerical simulation results", fontsize=16);
fig.tight_layout()
for extension in ("pdf", "png", "eps"):
    fig.savefig(output_path.with_suffix("." + extension), dpi=300,
        bbox_inches="tight")
plt.close(fig)
def run_numerical_full_pipeline(output_dir: Path) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    generate_swap_test_detection_data(output_dir / "swap_test_detection.csv")
    generate_protocol_acceptance_data(output_dir / "protocol_acceptance.csv")
    generate_repudiation_data(output_dir /
        "alice_repudiation_probability.csv")
    generate_resource_data(output_dir / "resource_scaling.csv")
    generate_complete_numerical_figure(output_dir, output_dir /
        "combined_numerical_simulations")
if __name__ == "__main__":

```

```

parser = argparse.ArgumentParser(description="Run the complete
    protocol-level numerical pipeline.")
parser.add_argument("--output-dir", type=Path,
    default=Path("numerical_pipeline_results"))
arguments = parser.parse_args();
run_numerical_full_pipeline(arguments.output_dir)

```

Appendix C Python Code for Fig. 6

The following Python code generates all four subfigures presented in Fig. 6.

```

"""Complete IBM/Qiskit circuit, retrieval, evaluation, and
    guarded-submission code."""
from __future__ import annotations
import argparse
import csv
import hashlib
import json
import math
from math import sqrt
from pathlib import Path
from qiskit import ClassicalRegister, QuantumCircuit, QuantumRegister,
    transpile
from qiskit.qasm3 import dumps as qasm3_dumps
from qiskit.transpiler import generate_preset_pass_manager
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2
# 1. Bell resources
def build_bell_pair_circuit(name='bell_pair'):
    q = QuantumRegister(2, 'bell')
    qc = QuantumCircuit(q, name=name)
    qc.h(q[0])
    qc.cx(q[0], q[1])
    return qc
def build_three_bell_pair_circuit():
    q = QuantumRegister(6, 'bell')
    qc = QuantumCircuit(q, name='three_v7_bell_pairs')
    for a, b in ((0, 1), (2, 3), (4, 5)):
        qc.h(q[a])
        qc.cx(q[a], q[b])
    qc.metadata = {'pairs': {'AB_S2': [0, 1], 'BT_S4': [2, 3], 'TB_S6': [4,
        5]}}
    return qc
# 2. Teleportation
def coherent_teleportation(qc, message, bell_sender, bell_receiver, label):
    qc.barrier(label=f'{label}_Bell_measurement')
    qc.cx(message, bell_sender)
    qc.h(message)
    qc.cx(bell_sender, bell_receiver)
    qc.cz(message, bell_receiver)
def build_teleportation_circuit(input_state='0'):
    qc = QuantumCircuit(3, 1, name=f'teleport_{input_state}')
    if input_state == '1':
        qc.x(0)
    elif input_state == '+':
        qc.h(0)
    elif input_state == '-':
        qc.x(0)
        qc.h(0)
    qc.h(1)
    qc.cx(1, 2)
    coherent_teleportation(qc, 0, 1, 2, 'S2')
    if input_state in {'+', '-'}:
        qc.h(2)
    qc.measure(2, 0)
    return qc

```

```

def build_three_teleportation_rounds(input_state='0'):
    qc = QuantumCircuit(7, 1, name='three_teleportation_rounds')
    if input_state == '1':
        qc.x(0)
    elif input_state == '+':
        qc.h(0)
    for a, b in ((1, 2), (3, 4), (5, 6)):
        qc.h(a)
        qc.cx(a, b)
    coherent_teleportation(qc, 0, 1, 2, 'S2')
    coherent_teleportation(qc, 2, 3, 4, 'S4')
    coherent_teleportation(qc, 4, 5, 6, 'S6')
    if input_state == '+':
        qc.h(6)
    qc.measure(6, 0)
    return qc

# 3. Strengthened-QOTP
def append_qotp(qc, qubit, key4, label='E_K'):
    U = encryption_unitary(key4)
    qc.unitary(U, [qubit], label=label)
def append_qotp_inverse(qc, qubit, key4, label='E_K^-1'):
    U = encryption_unitary(key4)
    qc.unitary(U.conj().T, [qubit], label=label)
def build_qotp_roundtrip_circuit(key4='0101', input_state='0'):
    qc = QuantumCircuit(1, 1, name='strengthened_qotp_roundtrip')
    if input_state == '1':
        qc.x(0)
    elif input_state == '+':
        qc.h(0)
    elif input_state == '-':
        qc.x(0)
        qc.h(0)
    append_qotp(qc, 0, key4)
    append_qotp_inverse(qc, 0, key4)
    if input_state in {'+', '-'}:
        qc.h(0)
    qc.measure(0, 0)
    return qc

# 4. Encrypted Bell records
def append_encrypted_record_roundtrip(qc, carrier, key4, record, label):
    if record[1] == '1':
        qc.x(carrier)
    if record[0] == '1':
        qc.z(carrier)
    append_qotp(qc, carrier, key4, f'{label}_encrypt')
    append_qotp_inverse(qc, carrier, key4, f'{label}_decrypt')
    if record[0] == '1':
        qc.z(carrier)
    if record[1] == '1':
        qc.x(carrier)
def build_encrypted_bell_record_circuit(key4='1010', record='01'):
    qc = QuantumCircuit(2, 2, name='encrypted_bell_record_roundtrip')
    qc.h(0)
    qc.cx(0, 1)
    append_encrypted_record_roundtrip(qc, 1, key4, record, 'E_KU_X')
    qc.cx(0, 1)
    qc.h(0)
    qc.measure([0, 1], [0, 1])
    return qc

# 5. Repeated swap tests
def build_repeated_swap_test_circuits(left='0', right='1', lam=8):
    return [build_swap_test_circuit(left, right, name=f'swap_{index +
1}_of_{lam}')] for index in range(lam)]

```

```

# 6. Staged components
def build_staged_components():
    return {'S2': build_teleportation_circuit(), 'S3_record':
            build_encrypted_bell_record_circuit(), 'S5_QOTP':
            build_qotp_roundtrip_circuit(), 'S6': build_teleportation_circuit()}
# 7. Composed staged-emulation candidate
FIXED_KEY = '0101'
def _prepare_message_copies(qc, qubits, input_state):
    for q in qubits:
        if input_state == '1':
            qc.x(q)
        elif input_state == '+':
            qc.h(q)
        elif input_state == '-':
            qc.x(q)
            qc.h(q)
def _swap_test(qc, anc, left, right, label):
    qc.barrier(label=label)
    qc.h(anc)
    qc.cswap(anc, left, right)
    qc.h(anc)
def build_composed_staged_emulation_candidate(input_state='0'):
    q = QuantumRegister(9, 'v7')
    c = ClassicalRegister(3, 'out')
    qc = QuantumCircuit(q, c, name='composed_staged_emulation_candidate')
    _prepare_message_copies(qc, (0, 1, 2), input_state)
    qc.barrier(label='S1_three_message_copies')
    for a, b, label in ((3, 4, 'AB_S2'), (5, 6, 'BT_S4'), (7, 8, 'TB_S6')):
        qc.h(a)
        qc.cx(a, b)
        qc.barrier(label=f'I2_{label}')
    coherent_teleportation(qc, 0, 3, 4, 'S2')
    append_encrypted_record_roundtrip(qc, 3, FIXED_KEY, '01', 'S3_X1_record')
    coherent_teleportation(qc, 4, 5, 6, 'S4')
    append_encrypted_record_roundtrip(qc, 5, FIXED_KEY, '10', 'S4_X2_record')
    qc.reset(0)
    _swap_test(qc, 0, 6, 2, 'S5_swap_test')
    coherent_teleportation(qc, 6, 7, 8, 'S6')
    append_encrypted_record_roundtrip(qc, 7, FIXED_KEY, '11', 'S6_X3_record')
    qc.reset(4)
    _swap_test(qc, 4, 8, 1, 'S7_swap_test')
    if input_state in {'+', '-':
        qc.h(8)
    qc.measure(0, 0)
    qc.measure(4, 1)
    qc.measure(8, 2)
    qc.metadata = {'candidate_type':
        'faithful_v7_three_explicit_bell_resources', 'success_predicate':
        'out=000', 'paper_resource_qubits': 9, 'swap_ancillas': 'reuse of
        consumed/dead measured-stage wires', 'not_implemented': ['QKD',
        'authentication', 'network separation', 'abstract arbitration']}
    return qc
def build_faithful_v7_unitary_reference(input_state='0'):
    qc = QuantumCircuit(11, name='faithful_v7_unitary_reference')
    _prepare_message_copies(qc, (0, 1, 2), input_state)
    for a, b in ((3, 4), (5, 6), (7, 8)):
        qc.h(a)
        qc.cx(a, b)
    coherent_teleportation(qc, 0, 3, 4, 'S2')
    append_encrypted_record_roundtrip(qc, 3, FIXED_KEY, '01', 'S3_X1_record')
    coherent_teleportation(qc, 4, 5, 6, 'S4')
    append_encrypted_record_roundtrip(qc, 5, FIXED_KEY, '10', 'S4_X2_record')
    _swap_test(qc, 9, 6, 2, 'S5_swap_test')

```

```

        coherent_teleportation(qc, 6, 7, 8, 'S6')
        append_encrypted_record_roundtrip(qc, 7, FIXED_KEY, '11', 'S6_X3_record')
        _swap_test(qc, 10, 8, 1, 'S7_swap_test')
        if input_state in {'+', '-'}:
            qc.h(8)
        return qc
def build_optimized_staged_candidate(input_state='0'):
    q = QuantumRegister(7, 'opt')
    c = ClassicalRegister(3, 'out')
    qc = QuantumCircuit(q, c, name='optimized_v7_staged_candidate')
    _prepare_message_copies(qc, (0, 1, 2), input_state)
    qc.barrier(label='S1')
    def leg(label):
        qc.h(3)
        qc.cx(3, 4)
        coherent_teleportation(qc, 0, 3, 4, label)
        qc.swap(0, 4)
        qc.reset(3)
        qc.reset(4)
    leg('S2')
    append_encrypted_record_roundtrip(qc, 6, FIXED_KEY, '01', 'S3_record')
    leg('S4')
    qc.reset(5)
    _swap_test(qc, 5, 0, 2, 'S5_swap_test')
    leg('S6')
    qc.reset(4)
    _swap_test(qc, 4, 0, 1, 'S7_swap_test')
    if input_state in {'+', '-'}:
        qc.h(0)
    qc.measure(5, 0)
    qc.measure(4, 1)
    qc.measure(0, 2)
    qc.metadata = {'candidate_type': 'optimized_staged_reset_reuse',
                    'success_predicate': 'out=000', 'not_paper_resource_count': True}
    return qc
# 8. IBM Runtime utilities
ROOT = Path(__file__).resolve().parents[1]
JOB_INDEX = ROOT /
    "figures_and_text/02_ibm_hardware_results/ibm_job_index.csv"
def runtime_service() -> QiskitRuntimeService:
    return QiskitRuntimeService(name="entropy-project")
def available_backends(service: QiskitRuntimeService):
    rows = []
    for backend in service.backends(operational=True, simulator=False):
        status = backend.status()
        rows.append({"backend": backend.name, "qubits": backend.num_qubits,
                     "operational": status.operational, "pending_jobs":
                     status.pending_jobs})
    return sorted(rows, key=lambda row: (row["pending_jobs"],
        -row["qubits"]))
def select_backend(service: QiskitRuntimeService, name: str = "ibm_fez"):
    backend = service.backend(name)
    if getattr(backend.configuration(), "simulator", True):
        raise RuntimeError("A real QPU is required.")
    return backend
def transpile_for_backend(circuit: QuantumCircuit, backend,
    optimization_level: int = 2):
    pass_manager = generate_preset_pass_manager(backend=backend,
        optimization_level=optimization_level)
    return pass_manager.run(circuit)
def extract_counts(pub_result) -> dict[str, int]:
    for name in dir(pub_result.data):
        if name.startswith("_"):

```

```

        continue
        value = getattr(pub_result.data, name)
        if hasattr(value, "get_counts"):
            return dict(value.get_counts())
        raise RuntimeError("No classical result register with get_counts() was
        found.")
def wilson_interval(successes: int, shots: int, z: float =
1.959963984540054) -> tuple[float, float]:
    p = successes / shots
    denominator = 1 + z * z / shots
    center = (p + z * z / (2 * shots)) / denominator
    half = z * sqrt(p * (1 - p) / shots + z * z / (4 * shots * shots)) /
    denominator
    return center - half, center + half
def read_job_index(path: Path = JOB_INDEX) -> list[dict[str, str]]:
    with path.open(encoding="utf-8", newline="") as handle:
        return list(csv.DictReader(handle))
def retrieve_existing_jobs(output_csv: Path) -> None:
    service = runtime_service(); rows = []
    for record in read_job_index():
        job = service.job(record["job_id"]); result = job.result()
        rows.append({"job_id": record["job_id"], "backend":
            record["backend"], "shots": record["shots"], "status":
            str(job.status()), "pub_count": len(result)})
    with output_csv.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]));
        writer.writeheader(); writer.writerows(rows)
def submit_one_candidate(shots: int, backend_name: str, confirm: bool) ->
str:
    if not confirm:
        raise RuntimeError("Submission requires both --submit and
        --confirm-submit.")
    service = runtime_service(); backend = select_backend(service,
        backend_name)
    circuit = build_composed_staged_emulation_candidate(); circuit.name =
        "composed_staged_emulation_candidate"
    isa = transpile_for_backend(circuit, backend, optimization_level=2)
    two_qubit = sum(count for name, count in isa.count_ops().items() if name
        in {"cz", "cx", "ecr", "rzz"})
    print(json.dumps({"backend": backend.name, "shots": shots, "qubits":
        isa.num_qubits, "depth": isa.depth(), "size": isa.size(),
        "two_qubit_gates": two_qubit}, indent=2))
    sampler = SamplerV2(mode=backend); job = sampler.run([isa], shots=shots)
    print(job.job_id()); return job.job_id()
def save_probability(counts: dict[str, int], expected: str, output_csv:
Path) -> None:
    shots = sum(counts.values()); success = counts.get(expected, 0); low,
        high = wilson_interval(success, shots)
    row = {"expected": expected, "shots": shots, "success_count": success,
        "correct_output_probability": success / shots, "wilson_95_low": low,
        "wilson_95_high": high, "counts_json": json.dumps(counts,
        sort_keys=True)}
    with output_csv.open("w", newline="", encoding="utf-8") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(row));
        writer.writeheader(); writer.writerow(row)
def main() -> None:
    parser = argparse.ArgumentParser(description="Retrieve recorded IBM jobs
        or explicitly submit one candidate.")
    parser.add_argument("--mode", choices=["retrieve_existing_jobs",
        "list_backends"], default="retrieve_existing_jobs")
    parser.add_argument("--output", type=Path,
        default=Path("retrieved_jobs.csv"))

```

```

parser.add_argument("--backend", default="ibm_fez");
    parser.add_argument("--shots", type=int, default=128)
parser.add_argument("--submit", action="store_true");
    parser.add_argument("--confirm-submit", action="store_true")
args = parser.parse_args()
if args.submit:
    submit_one_candidate(args.shots, args.backend, args.confirm_submit)
elif args.mode == "list_backends":
    print(json.dumps(available_backends(runtime_service()), indent=2))
else:
    retrieve_existing_jobs(args.output); print(args.output)
import csv
import matplotlib.pyplot as plt
from qiskit import qasm3, transpile
from qiskit.qasm3 import dumps as qasm3_dumps
# Self-contained matrix definitions used by the strengthened-QOTP circuit.
import numpy as np
X_MATRIX = np.array([[0, 1], [1, 0]], dtype=complex)
Y_MATRIX = np.array([[0, -1j], [1j, 0]], dtype=complex)
Z_MATRIX = np.array([[1, 0], [0, -1]], dtype=complex)
T_STRENGTHENED = np.sqrt(1j / 3) * (X_MATRIX - Y_MATRIX + Z_MATRIX)
def encryption_unitary(key4):
    """Return the one-qubit strengthened-QOTP unitary for a four-bit key
    block."""
    if len(key4) != 4 or set(key4) - {"0", "1"}:
        raise ValueError("key block must contain four binary digits")
    a, b, c, d = map(int, key4)
    return (
        np.linalg.matrix_power(X_MATRIX, a)
        @ np.linalg.matrix_power(Z_MATRIX, b)
        @ T_STRENGTHENED
        @ np.linalg.matrix_power(X_MATRIX, c)
        @ np.linalg.matrix_power(Z_MATRIX, d)
    )
# Circuit collection used for the modular and complete folders.
def build_swap_test_verification_circuit(left="0", right="1"):
    qc = QuantumCircuit(3, 1, name="swap_test_verification")
    if left == "1":
        qc.x(1)
    elif left == "+":
        qc.h(1)
    elif left == "-":
        qc.x(1); qc.h(1)
    if right == "1":
        qc.x(2)
    elif right == "+":
        qc.h(2)
    elif right == "-":
        qc.x(2); qc.h(2)
    qc.h(0); qc.cswap(0, 1, 2); qc.h(0); qc.measure(0, 0)
    return qc
def build_swap_test_circuit(left="0", right="1", name="swap_test"):
    """Compatibility wrapper used by the repeated swap-test circuit
    builder."""
    circuit = build_swap_test_verification_circuit(left, right)
    circuit.name = name
    return circuit
def circuit_collection():
    staged = build_staged_components()
    modular = {
        "Bell_Pair": build_bell_pair_circuit(),
        "Three_Bell_Pairs": build_three_bell_pair_circuit(),
        "Teleportation": build_teleportation_circuit(),

```

```

        "Three_Teleportation_Rounds": build_three_teleportation_rounds(),
        "Strengthened_QOTP_Roundtrip": build_qotp_roundtrip_circuit(),
        "Encrypted_Bell_Record": build_encrypted_bell_record_circuit(),
        "Swap_Test_Verification": build_swap_test_verification_circuit(),
        "Stage_S2": staged["S2"],
        "Stage_S3_Record": staged["S3_record"],
        "Stage_S5_QOTP": staged["S5_QOTP"],
        "Stage_S6": staged["S6"],
    }
    complete = {
        "Composed_Full_Candidate":
            build_composed_staged_emulation_candidate(),
        "Optimized_Staged_Candidate": build_optimized_staged_candidate(),
    }
    return modular, complete
def save_circuit(circuit, base_path):
    base_path.parent.mkdir(parents=True, exist_ok=True)
    (base_path.with_suffix(".qasm")).write_text(qasm3_dumps(circuit),
        encoding="utf-8")
    (base_path.with_suffix(".txt")).write_text(str(circuit.draw("text",
        fold=160)), encoding="utf-8")
    figure = circuit.draw("mpl", style="iqp", fold=80, idle_wires=False)
    for extension in ("pdf", "png", "svg"):
        figure.savefig(base_path.with_suffix("." + extension), dpi=300 if
            extension == "png" else None, bbox_inches="tight",
            facecolor="white")
    plt.close(figure)
def generate_ibm_target_circuits(backend_name="ibm_fez"):
    # This function connects only to obtain the backend target; it never
    # submits a job.
    import os
    root = Path(__file__).resolve().parent
    try:
        service = QiskitRuntimeService(name="entropy-project")
    except Exception:
        # Credentials remain in environment variables and are never written
        # to output.
        service = QiskitRuntimeService(
            channel="ibm_quantum_platform",
            token=os.environ["QISKIT_IBM_TOKEN"],
            instance=os.environ["QISKIT_IBM_CRN"],
        )
    backend = service.backend(backend_name)
    if getattr(backend.configuration(), "simulator", True):
        raise RuntimeError("A real QPU target is required.")
    modular, complete = circuit_collection()
    rows = []
    for group, circuits in (("Modular_Circuits", modular),
        ("Complete_Circuits", complete)):
        for name, logical in circuits.items():
            pass_manager = generate_preset_pass_manager(backend=backend,
                optimization_level=1)
            isa = pass_manager.run(logical)
            save_circuit(isa, root / group / name)
            two_qubit = sum(count for gate, count in isa.count_ops().items()
                if gate in {"cz", "cx", "ecr", "rzz"})
            rows.append({
                "group": group, "circuit": name, "backend": backend.name,
                "simulator": False, "physical_qubits": isa.num_qubits,
                "depth": isa.depth(), "size": isa.size(), "two_qubit_gates":
                    two_qubit,
            })
    })

```

```

with (root / "IBM_Target_Transpilation_Metrics.csv").open("w",
    newline="", encoding="utf-8") as handle:
    writer = csv.DictWriter(handle, fieldnames=list(rows[0]));
    writer.writeheader(); writer.writerows(rows)
if False and __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Generate target-transpiled
        circuit diagrams without submitting a job.")
    parser.add_argument("--backend", default="ibm_fez")
    args = parser.parse_args()
    generate_ibm_target_circuits(args.backend)
# Existing IBM result-data processing and figure generation
import pandas as pd
PACKAGE_ROOT = Path(__file__).resolve().parents[2]
IBM_DATA_ROOT = PACKAGE_ROOT / "01_Figure_Code/IBM_Hardware_Results"
def generate_recorded_ibm_hardware_figure(output_path: Path) -> None:
    """Plot recorded IBM results. This function does not connect to IBM or
        submit jobs."""
    teleportation = pd.read_csv(IBM_DATA_ROOT / "teleportation_hardware.csv")
    qotp = pd.read_csv(IBM_DATA_ROOT / "qotp_hardware.csv")
    swap = pd.read_csv(IBM_DATA_ROOT / "swap_test_hardware.csv")
    aer = pd.read_csv(IBM_DATA_ROOT / "composed_candidate_aer.csv").iloc[0]
    pilot = pd.read_csv(IBM_DATA_ROOT /
        "composed_candidate_pilot.csv").iloc[0]
    final = pd.read_csv(IBM_DATA_ROOT /
        "composed_candidate_final.csv").iloc[0]
    fig, axes = plt.subplots(2, 2, figsize=(12, 9))
    axes[0, 0].bar(teleportation["input_state"],
        teleportation["correct_output_probability"], color="0.35")
    axes[0, 0].set(title="(a) Quantum teleportation", ylabel="Correct-output
        probability", ylim=(0, 1.05))
    states = ["0", "1", "+", "-"]; positions = np.arange(4); width = 0.18
    keys = [str(value) for value in sorted(qotp["key"].unique())[:4]]
    for index, key in enumerate(keys):
        part = qotp[qotp["key"].astype(str) ==
            key].set_index("input_state").reindex(states)
        axes[0, 1].bar(positions + (index - 1.5) * width,
            part["correct_output_probability"], width, label=f"K={key}")
    axes[0, 1].set_xticks(positions, states); axes[0, 1].set(title="(b)
        Strengthened-QOTP roundtrip", ylabel="Correct-output probability",
        ylim=(0.94, 1.005)); axes[0, 1].legend(fontsize=8)
    cases = ["identical_0_0", "identical_plus_plus", "orthogonal_0_1",
        "orthogonal_plus_minus", "fixed_overlap_F_0.25"]
    selected = swap[(swap["execution"] == "Final") & (swap["lambda"] ==
        1)].set_index("case").loc[cases]
    ideal = (1 - selected["fidelity"].to_numpy()) / 2
    axes[1, 0].bar(np.arange(5) - 0.18, ideal, 0.36, label="Ideal",
        color="0.8", edgecolor="black")
    axes[1, 0].bar(np.arange(5) + 0.18,
        selected["single_test_detection_probability"], 0.36, label="IBM
        hardware", color="0.25")
    axes[1, 0].set_xticks(range(5), ["0/0", "+/+ ", "0/1", "+/- ", "F=.25"]);
    axes[1, 0].set(title="(c) Swap-test verification",
        ylabel="Single-test detection probability", ylim=(0, 0.65)); axes[1,
        0].legend(fontsize=8)
    values = [aer["success_probability"], pilot["success_probability"],
        final["success_probability"]]
    axes[1, 1].bar(["Aer ideal", "Pilot", "Final"], values, color=["0.75",
        "0.45", "0.15"])
    axes[1, 1].set(title="(d) Composed staged-emulation candidate",
        ylabel="Combined correct-output probability", ylim=(0, 1.05))
    fig.suptitle("IBM Quantum hardware results", fontsize=16);
    fig.tight_layout()
    output_path.parent.mkdir(parents=True, exist_ok=True)

```

```

for extension in ("pdf", "png", "eps"):
    fig.savefig(output_path.with_suffix("." + extension), dpi=300,
                bbox_inches="tight")
plt.close(fig)
def main_complete_ibm_code() -> None:
    parser = argparse.ArgumentParser(description="Process existing IBM
    results; connection and submission require explicit flags.")
    parser.add_argument("--mode", choices=["plot-existing",
    "retrieve-existing", "list-backends", "generate-target-circuits"],
    default="plot-existing")
    parser.add_argument("--output", type=Path,
    default=Path("ibm_hardware_results"))
    parser.add_argument("--backend", default="ibm_fez");
    parser.add_argument("--shots", type=int, default=128)
    parser.add_argument("--submit", action="store_true");
    parser.add_argument("--confirm-submit", action="store_true")
    args = parser.parse_args()
    if args.submit:
        submit_one_candidate(args.shots, args.backend, args.confirm_submit)
    elif args.mode == "retrieve-existing":
        retrieve_existing_jobs(args.output.with_suffix(".csv"))
    elif args.mode == "list-backends":
        print(json.dumps(available_backends(runtime_service()), indent=2))
    elif args.mode == "generate-target-circuits":
        generate_ibm_target_circuits(args.backend)
    else:
        generate_recorded_ibm_hardware_figure(args.output)
if __name__ == "__main__":
    main_complete_ibm_code()

```

References

- Shi, R. H.; Feng, Y. Y.; Shi, J. J. Arbitrated Quantum Signature Scheme with Quantum Walks on Regular Graphs. *J. Electron. Inf. Technol.* **2020**, *42*, 89–97.
- Shang, T.; Hou, P. L.; Liu, Y. C.; Xiong, K. Y.; Liu, J. W. Arbitrated Quantum (t,n) Threshold Group Signature Scheme Based on Multi-coin Quantum Walk. *Sci. Sin. Phys. Mech. Astron.* **2024**, *54*, 240316.
- Shi, R. H.; Zhang, M. W. Privacy-preserving Quantum Sealed-bid Auction Based on Grover's Search Algorithm. *Sci. Rep.* **2019**, *9*, 7626.
- Yoon, C. S.; Kang, M. S.; Lim, J. I.; Yang, H. J. Quantum Signature Scheme Based on a Quantum Search Algorithm. *Phys. Scr.* **2015**, *90*, 015103.
- Grover, L. K. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing*, Philadelphia, PA, USA, 22–24 May 1996; pp. 212–219.
- Yoon, C. S.; Hong, C. H.; Kang, M. S.; Choi, J. W.; Yang, H. J. Quantum Asymmetric Key Crypto Scheme Using Grover Iteration. *Sci. Rep.* **2023**, *13*, 3810.
- Lu, D.; Li, Z.; Yu, J.; Han, Z. A Verifiable Arbitrated Quantum Signature Scheme Based on Controlled Quantum Teleportation. *Entropy* **2022**, *24*, 111.
- Zeng, G.; Keitel, C. H. Arbitrated-Quantum Signature Scheme. *Phys. Rev. A* **2002**, *65*, 042312.
- Li, Q.; Chan, W. H.; Long, D. Y. Arbitrated Quantum Signature Scheme Using Bell States. *Phys. Rev. A* **2009**, *79*, 054307.
- Pang, Z.; Xiang, H. A Simplified Arbitrated Quantum Signature Protocol Without Entanglement: Design Premise and Security Analysis. *Opt. Commun.* **2025**, *595*, 132315.
- Zou, X.; Qiu, D. Security Analysis and Improvements of Arbitrated Quantum Signature Schemes. *Phys. Rev. A* **2010**, *82*, 042325.
- Yang, Y. G.; Zhou, Z.; Teng, Y. W.; Wen, Q. Y. Arbitrated Quantum Signature with an Untrusted Arbitrator. *Eur. Phys. J. D* **2011**, *61*, 773–778.

13. Li, Q.; Li, C.; Long, D.; Chan, W. H.; Wang, C. Efficient Arbitrated Quantum Signature and Its Proof of Security. *Quantum Inf. Process.* **2013**, *12*, 2427–2439. 2375
14. Choi, J. W.; Chang, K. Y.; Hong, D. Security Problem on Arbitrated Quantum Signature Schemes. *Phys. Rev. A* **2011**, *84*, 062330. 2376
15. Zhang, K. J.; Zhang, W. W.; Li, D. Improving the Security of Arbitrated Quantum Signature Against the Forgery Attack. *Quantum Inf. Process.* **2013**, *12*, 2655–2669. 2377
16. Kim, T.; Choi, J. W.; Jho, N. S.; Lee, S. Quantum Messages with Signatures Forgeable in Arbitrated Quantum Signature Schemes. *Phys. Scr.* **2015**, *90*, 025101. 2378
17. Boykin, P. O.; Roychowdhury, V. Optimal Encryption of Quantum Bits. *Phys. Rev. A* **2003**, *67*, 042317. 2379
18. Leung, D. W. Quantum Vernam Cipher. arXiv:quant-ph/0012077, 2000. 2380
19. Bennett, C. H.; Brassard, G. Quantum Cryptography: Public Key Distribution and Coin Tossing. In *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, Bangalore, India, 9–12 December 1984; pp. 175–179. 2381
20. Ekert, A. K. Quantum Cryptography Based on Bell's Theorem. *Phys. Rev. Lett.* **1991**, *67*, 661–663. 2382
21. Bennett, C. H. Quantum Cryptography Using Any Two Nonorthogonal States. *Phys. Rev. Lett.* **1992**, *68*, 3121–3124. 2383
22. Mayers, D. Unconditional Security in Quantum Cryptography. *J. ACM* **2001**, *48*, 351–406. 2384
23. Lo, H. K. A Simple Proof of the Unconditional Security of Quantum Key Distribution. *J. Phys. A: Math. Gen.* **2001**, *34*, 6957. 2385
24. Lu, H.; Guo, G. C. Teleportation of a Two-Particle Entangled State via Entanglement Swapping. *Phys. Lett. A* **2000**, *276*, 209–212. 2386
25. Lo, H. K.; Chau, H. F. Unconditional Security of Quantum Key Distribution over Arbitrarily Long Distances. *Science* **1999**, *283*, 2050–2056. 2387
26. Shor, P. W.; Preskill, J. Simple Proof of Security of the BB84 Quantum Key Distribution Protocol. *Phys. Rev. Lett.* **2000**, *85*, 441–444. 2388
27. Abruzzo, S.; Bratzik, S.; Bernardes, N. K.; Kampermann, H.; van Loock, P.; Bruß, D. Quantum Repeaters and Quantum Key Distribution: Analysis of Secret-Key Rates. *Phys. Rev. A* **2013**, *87*, 052315. 2389
28. Barnum, H.; Crépeau, C.; Gottesman, D.; Smith, A.; Tapp, A. Authentication of Quantum Messages. In *Proceedings of the 43rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Vancouver, BC, Canada, 16–19 November 2002; pp. 449–458. 2390